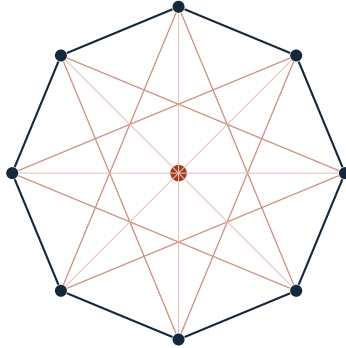




The Laplacian Reader

Graphs, differential operators, generative constraint
and the hand that moves them

Seven questions, one operator

Gosford, New South Wales
July 25, 2026

How to read this

This reader began as a set of apparently unrelated questions — about databases, differential equations, Fourier analysis, language models, Arabic and Latin calligraphy, procedural generation, gestural interfaces, dithering, cellular automata and one-bit rendering.

They are not unrelated. Most of them are the same mathematical object approached from different directions; the rest are the machinery you would use to build the others.

The object is the **Laplacian**.

A graph carries a native differential calculus. Its Laplacian $\mathbf{L} = \mathbf{B}^T \mathbf{B} = \mathbf{D} - \mathbf{A}$ is simultaneously:

- the discretisation of ∇^2 , so *PDEs become graph computation*;
- the operator whose eigenbasis is the Fourier basis, so *frequency is a graph-theoretic notion*;
- the diffusion generator that message-passing networks integrate, so *deep learning on graphs is numerical PDE solving*;
- the constraint structure that generative algorithms propagate over, so *procedural generation is constraint satisfaction on a graph*;
- the operator whose eigenvalues a reaction–diffusion system selects between, so *pattern formation is eigenvector selection*.

Chapters 8 and 9 run a second thread alongside it — **error, and where you choose to put it** — which turns out to connect printing, signal processing, theoretical biology and glitch art to the same page.

The chapters have been reordered from the original questions so the argument builds. Nothing has been dropped. If you want to read in the order you asked, use the map below.

Original question	Ch.	What it turns out to be
Are all databases graphs?	1	A question about arity and access paths, not data models
Graphs and PDEs?	2	The central chapter. L is $-\nabla^2$
And Fourier transforms?	3	The eigenbasis of the same operator
Are LLMs made of paired JSON?	4	No — and attention is a learned adjacency matrix
Encoding blackletter / square Kufic as graphs	5	Two opposite problems: discrete CSP vs. continuous constraint solve
Does WFC use graphs?	6	WFC is arc consistency on a constraint graph
Hands, video, TouchDesigner	7	Dataflow DAGs, and a filter-design problem in disguise
Dithering, halftones, cellular automata, one-bit graphics	8	Spectral shaping of quantisation error; automata are message passing
Rendering video and data artistically	9	Every style is a policy for where the error goes

Conventions

Each chapter ends with its own glossary. Every term from every chapter is collated alphabetically in the MASTER GLOSSARY at the back, tagged with the chapter of origin.

Four kinds of box appear throughout:

The load-bearing idea of a section. If you read nothing else on the page, read this.

Sanity check — A claim you can verify yourself with a small computation, rather than take on authority. These are deliberate — the arguments here are checkable and you should check them.

The gotcha — A failure mode that will consume the project if it is not anticipated. These are not warnings in general; each one is a specific mechanism with a specific mitigation.

At the bench — Practical build notes — what to reach for, what it costs, what it does badly.

All figures are vector (TIKZ/PGFPLLOTS), generated from source alongside the text. Metric units and Australian spelling throughout.

Contents

How to read this	i
1 Are All Databases Graphs?	1
1.1 The formal reduction, and what it costs	1
1.2 Access paths: joins versus pointers	2
1.3 The expressiveness gap is real	3
1.4 The containment lattice	3
Glossary: Databases, Relations and Graphs	5
2 The Laplacian: Graphs as Discrete Manifolds	6
2.1 A graph has a gradient	6
2.2 Now just write the PDEs down	7
2.3 The correspondence runs both ways	8
2.3.1 Discretisation produces a graph	8
2.3.2 Sampling a manifold produces a graph that converges to it	8
2.4 The payoff: GNNs are PDE solvers	10
Glossary: The Laplacian: Graphs as Discrete Manifolds	11
3 Fourier: The Eigenbasis of the Laplacian	12
3.1 What the Fourier basis actually is	12
3.2 The check: the DFT is the GFT on a ring	13
3.3 Why there is no fast graph Fourier transform	14
3.3.1 The workaround: polynomials in L	14
3.4 The unification	15
Glossary: Fourier: The Eigenbasis of the Laplacian	17
4 Language Models: What the Data Actually Is	18
4.1 Three stages, three different data shapes	18
4.1.1 Pretraining: no pairs at all	18
4.1.2 SFT: paired-looking, but flattened	18
4.1.3 Preference tuning: triples, not pairs	19
4.2 Attention is a learned adjacency matrix	20
4.3 Positional encoding closes the loop	20
Glossary: Language Models: What the Data Actually Is	22
5 Encoding Script: Kufic and Blackletter	23
5.1 Square Kufic: a discrete CSP on a grid graph	23
5.1.1 Layer A: the orthographic FSA	23
5.1.2 Layer B: the geometric CSP	24
5.2 Blackletter: continuous constraints, and why gridding it is wrong	25
5.2.1 The right structure: a part graph	25
5.3 The unifying move: don't encode rules <i>in</i> a graph	26

Glossary: Encoding Script: Kufic and Blackletter as Graphs	28
6 Wave Function Collapse as Constraint Solving	30
6.1 The translation	30
6.2 Why contradictions happen, and why restarting is a choice	30
6.3 Your Sudoku case, precisely	31
6.4 WFC on arbitrary graphs	32
6.5 Where WFC sits in the family	33
Glossary: Wave Function Collapse and Procedural Generation	35
7 Dataflow, TouchDesigner and Hands	36
7.1 TouchDesigner is a typed dataflow DAG	36
7.1.1 The distinction that actually matters	37
7.2 Hands from video: the pipeline	37
7.3 The real problem is filter design	37
7.4 Discrete decisions need hysteresis	39
7.5 The honest design constraint	39
Glossary: Dataflow, TouchDesigner and Hands	41
8 One Bit: Dithering, Halftones and Automata	42
8.1 Four quantisers, one test image	42
8.2 Error diffusion is a filter on a directed graph	42
8.2.1 Ordered dithering: stateless, parallel, spiky	42
8.3 Cellular automata are message passing	45
8.4 Turing patterns select a Laplacian eigenvector	45
8.5 Why one bit produces such a strong aesthetic	47
Glossary: One Bit: Dithering, Halftones and Automata	48
9 Rendering: Putting the Error Where You Want It	50
9.1 The temporal axis, where naive implementations die	50
9.2 The design space	51
9.3 Three unusual routes, spelled out	51
9.3.1 WFC as a renderer	52
9.3.2 Reaction–diffusion driven by the image	53
9.3.3 Datamoshing, mechanically	53
9.4 It does not have to be an image	53
9.5 The seams	54
Glossary: Rendering: Putting the Error Where You Want It	55
Master Glossary	56
Sources and Further Reading	62

List of Figures

1.1	Reification of an n -ary tuple into a star graph	1
1.2	Access paths: index probe versus index-free adjacency	2
1.3	The containment lattice of data models	4
2.1	The Laplacian as $\mathbf{B}^T \mathbf{B}$, worked example	7
2.2	Heat diffusion on a ring graph	8
2.3	Finite-difference stencil, grid graph, and sparsity pattern	9
2.4	From manifold to point cloud to convergent graph Laplacian	9
3.1	The ring graph spectrum against the closed form	13
3.2	Laplacian eigenvectors of a path graph	14
3.3	Chebyshev order equals hop radius	15
3.4	The Laplacian as the common object	16
4.1	The three training stages and their data shapes	19
4.2	Loss masking over a flattened chat stream	19
4.3	Attention as a masked, learned adjacency matrix	20
5.1	Arabic contextual joining as a finite automaton	24
5.2	Valid square Kufic and its two failure modes	25
5.3	The broad-nib model generates blackletter	26
5.4	The blackletter part graph	26
5.5	Graph grammar productions for both scripts	27
6.1	One WFC collapse-and-propagate cycle	31
6.2	The Sudoku constraint graph	32
6.3	WFC generalises to any graph	33
6.4	WFC's place among procedural generation methods	34
7.1	A typed TouchDesigner dataflow graph	36
7.2	MediaPipe's 21 hand landmarks	38
7.3	One Euro filter against a fixed low-pass	38
7.4	Chatter, hysteresis, and the pinch state machine	39
7.5	The hybrid input split	40
8.1	Four quantisers on the same gradient ramp	43
8.2	Radially averaged error spectra for four quantisers	43
8.3	The Floyd–Steinberg kernel as stencil, graph, and filter	44
8.4	Recursive construction of the Bayer threshold matrix	44
8.5	Cellular automata and message passing are the same update	45
8.6	The Turing dispersion relation	46
9.1	Three strategies for temporally coherent dithering	51

9.2	The error-routing design space	52
9.3	The one-bit rendering pipeline	54

CHAPTER 1

Are All Databases Graphs?

No — but the way the answer fails is more instructive than the answer.

1.1 The formal reduction, and what it costs

A graph is a set V together with a binary relation $E \subseteq V \times V$. That is the entire definition. In relational terms, a graph is *one table with two columns*.

This immediately suggests a reduction. Any n -ary relation can be **reified**: turn each tuple into a node, and emit n edges from that node to its attribute values, labelled by column name. Figure 1.1 shows the transformation. This is not a trick — it is precisely how RDF triple stores represent everything, and it means that in a formal sense every database *is* encodable as a graph.

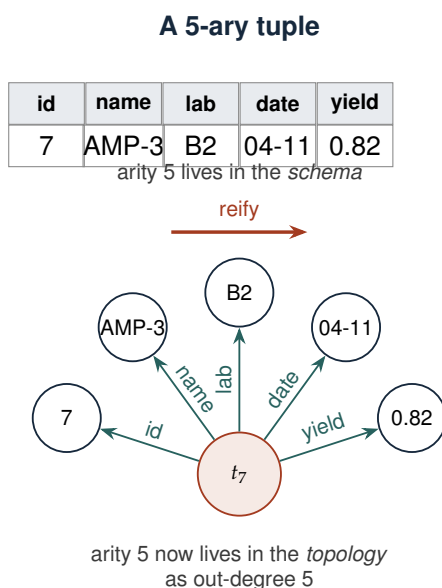


Figure 1.1. Reification. A 5-ary tuple becomes a node with five labelled edges. The information is preserved exactly, but arity has moved from the *schema* (where the query planner can see it and reason about it statically) into the *topology* (where it must be rediscovered at query time). This is the trade the reduction makes, and it is the reason “every database is a graph” is true and useless in equal measure.

So the reduction goes through. What it does not do is make the two systems equivalent in any sense you would care about while building something. The information survives; the *query planner’s* information does not.

The dividing line between database models is not the data model. It is the **access path** — the physical strategy by which the engine gets from one record to a related one, and the cost model that strategy induces.

1.2 Access paths: joins versus pointers

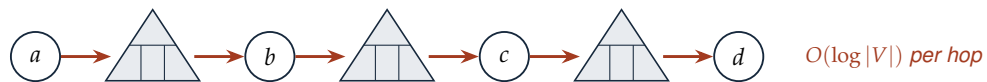
Relational. A relationship is a shared value. To follow it you *join*: probe an index, or hash-build and probe, or sort-merge. The cost is governed by cardinality and by index statistics, and it is largely independent of how many neighbours any particular record has. Following a relationship k steps is a k -way self-join.

Native graph. A relationship is a *pointer*. This is **index-free adjacency**: the edge record physically contains the address of the neighbouring node, so traversal is a dereference. The cost is governed by degree and by memory locality, and it does not depend on the size of the database at all.

Figure 1.2 makes the asymmetry concrete.

Relational

a 3-hop traversal is a 3-way self-join



Native graph

the same traversal is three dereferences



Cost driver: cardinality, selectivity, index statistics — properties of the *whole table*.

Failure mode: estimation error compounds *multi-plicatively* with depth, so plans degrade sharply past 3–4 hops.

Unbounded depth: requires WITH RECURSIVE; most planners cannot push predicates through the recursive term.

Cost driver: degree and memory locality — properties of the *local neighbourhood*. Independent of $|V|$.

Failure mode: fan-out. Cost is $O(\prod_i \bar{d}_i)$, so one high-degree hub explodes the frontier. *Supernodes* are the characteristic pathology.

Unbounded depth: native ($[: R^*]$).

Figure 1.2. The access path is the real distinction. Both engines can answer the same questions; they pay for them differently. The relational planner pays $O(\log |V|)$ per hop and compounds its estimation error with depth. The graph engine pays $O(1)$ per hop and is instead exposed to *fan-out* — a traversal through a high-degree hub node explodes just as badly, which is why supernodes are the characteristic graph-database pathology.

Sanity check — The asymmetry is easy to confirm at small scale. Load the same edge set into PostgreSQL and into any property-graph store, then measure a 2-hop and a 6-hop neighbourhood query. The 2-hop times will be comparable and the relational one may well win. The 6-hop times will not be comparable. One measurement settles the question for your data far better than any argument about models.

1.3 The expressiveness gap is real

Performance is the visible difference. There is also a genuine *logical* one, and it is worth stating precisely because it is often fudged.

Transitive closure is not expressible in first-order logic. It follows that it is not expressible in core relational algebra either. This is a theorem, not an implementation limitation.

Graph query languages express it natively. A Cypher variable-length pattern `((a)-[:KNOWS*]->(b))` or a SPARQL property path is a **regular path query**: it matches paths whose sequence of edge labels forms a word in a regular language. Since the Kleene star is available, transitive closure comes for free.

SQL's `WITH RECURSIVE`, standardised in SQL:1999, exists specifically to close this gap. It succeeds semantically. Operationally it remains awkward, because most planners cannot push predicates through the recursive term, so filtering happens after the fixpoint rather than during it.

1.4 The containment lattice

The models form a strict hierarchy, each level a constraint on the one above (Figure 1.3). Reading downwards, every level is a special case; reading upwards, every level is a generalisation that recovers something the level below had to encode by convention.

At the bench — The practical decision rule is not “is my data a graph” — almost all data is, under reification. It is: *do my queries have unbounded or deep traversal depth?* If the deepest question you ask is a 2-hop join, a relational engine with good indices will beat a graph engine on the same hardware, and you keep transactions, tooling and a query planner that has had four decades of work put into it. If you routinely ask questions whose depth is data-dependent (reachability, shortest path, pattern matching over paths of unknown length), the relational formulation is fighting the engine and the graph one is not.

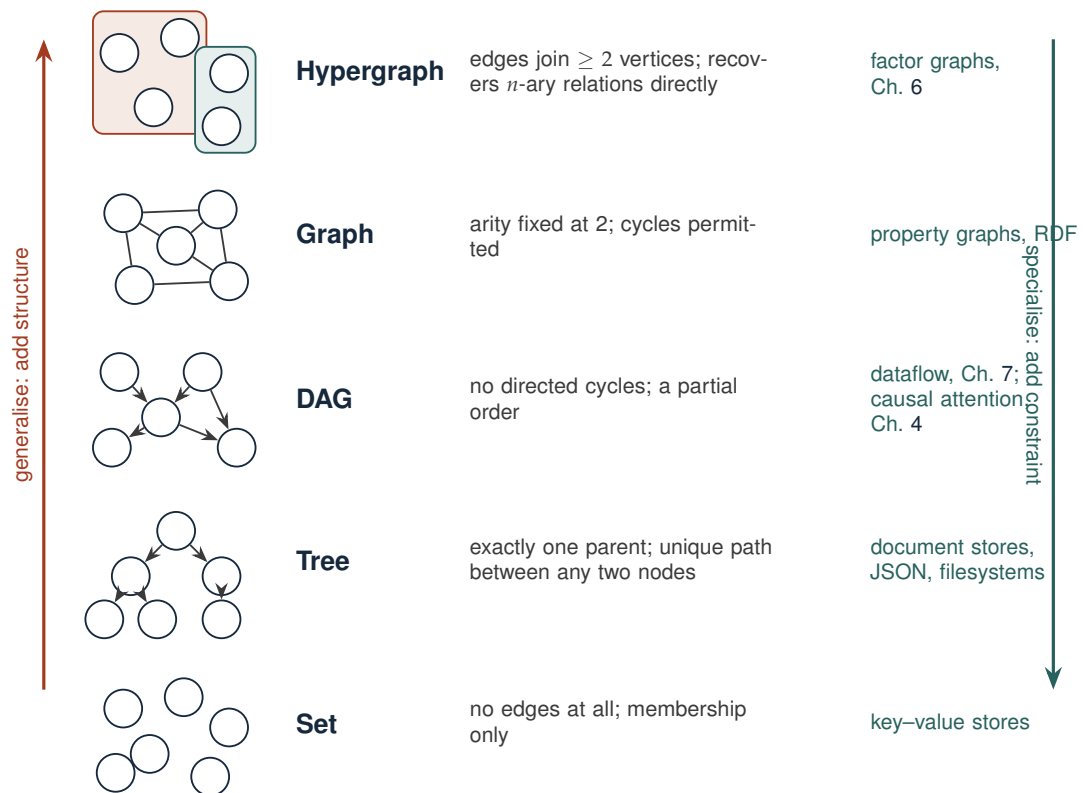


Figure 1.3. The containment lattice. Each model is the one above it plus a restriction. Choosing a database is choosing where on this lattice your problem actually sits — and the common failure is picking a level *below* the data, then reintroducing the missing structure by convention (foreign keys standing in for edges, path strings standing in for trees) where the engine cannot optimise it.

Glossary: Databases, Relations and Graphs

Terms introduced in this chapter

Relation — A subset of a Cartesian product $R \subseteq A_1 \times \dots \times A_n$. The whole of the relational model is relations plus operations on them.

Arity — The number of columns n in a relation. A graph is exactly the arity-2 case; everything distinctive about graph databases follows from fixing $n = 2$.

Reification — Rewriting an n -ary tuple as a node plus n binary edges to its attribute values. The mechanism by which any relational schema can be re-expressed as a graph, at the cost of moving arity out of the schema and into the topology.

RDF triple — A statement of the form (subject, predicate, object). A graph edge with a typed label; the storage format that makes reification the native representation.

Index-free adjacency — Storing each edge as a direct pointer from one record to its neighbour, so traversal costs $O(\text{degree})$ rather than $O(\log |V|)$ per hop. The single property that distinguishes a native graph engine from a graph API over tables.

Join — The relational operation combining tuples across relations on a shared attribute. A k -hop traversal expressed relationally is a k -way self-join.

Cardinality — The number of tuples in a relation, or the number of distinct values in a column. The primary input to a relational cost model, and the reason relational planners degrade on deep traversals: cardinality estimates compound multiplicatively with join depth.

Transitive closure — Given relation E , the relation E^+ containing (u, v) whenever a path $u \rightarrow v$ exists. Provably *not* expressible in first-order logic, hence not in core relational algebra.

Regular path query (RPQ) — A query matching paths whose edge-label sequence belongs to a regular language, e.g. `:knows*` in Cypher or a SPARQL property path. RPQs express transitive closure natively; this is a genuine expressiveness gap, not a performance one.

Recursive CTE — SQL's `WITH RECURSIVE` construct, added in SQL:1999 specifically to recover transitive closure. Semantically sufficient, operationally poor: the planner generally cannot push predicates through the recursion.

Property graph — The Neo4j/TinkerPop model: nodes and edges both carry a label and an arbitrary key-value map. Distinct from RDF, where attributes are themselves edges.

Hypergraph — A generalisation in which an edge may connect any number of vertices. Sits above the graph in the containment lattice and recovers n -ary relations without reification.

CHAPTER 2

The Laplacian: Graphs as Discrete Manifolds

This is the central chapter. Everything after it is a consequence of it.

The connection between graphs and partial differential equations is not an analogy, a metaphor, or a modelling convenience. A graph *carries a differential calculus natively*, and the operator at the centre of that calculus is the Laplacian. Once that is set up, the heat equation, the wave equation, Poisson's equation, Fourier analysis, spectral clustering, manifold learning and graph neural networks all fall out of the same object.

2.1 A graph has a gradient

Let $G = (V, E)$ have $|V| = n$ vertices and $|E| = m$ edges. Fix an arbitrary orientation on each edge — arbitrary because, as we will see, the operator we build is independent of the choice.

Define two function spaces:

$$\begin{aligned} C^0 &= \{f : V \rightarrow \mathbb{R}\} \cong \mathbb{R}^n && 0\text{-forms} \text{ — functions on vertices} \\ C^1 &= \{\omega : E \rightarrow \mathbb{R}\} \cong \mathbb{R}^m && 1\text{-forms} \text{ — functions on edges} \end{aligned}$$

The **incidence matrix** $\mathbf{B} \in \mathbb{R}^{m \times n}$ has, in the row for edge $e = (u \rightarrow v)$, the entry -1 in column u and $+1$ in column v . Then

$$(\mathbf{B}f)_e = f(v) - f(u)$$

which is a difference across an edge. That is a discrete derivative — and not by analogy. $\mathbf{B}$ is the discrete exterior derivative d acting on 0-forms: the graph's gradient operator.

Its transpose $\mathbf{B}^\top : C^1 \rightarrow C^0$ sums signed edge values at each vertex — net outflow, which is the discrete divergence. Composing:

$$\mathbf{L} = \mathbf{B}^\top \mathbf{B} = \mathbf{D} - \mathbf{A}$$

The graph Laplacian is $\text{div} \circ \text{grad}$, which is precisely what ∇^2 is. With this sign convention $\mathbf{L}$ is *positive* semi-definite, so it corresponds to $-\nabla^2$.

Four properties follow from the factorisation alone, with no computation:

- $\mathbf{L}$ is symmetric positive semi-definite, since $x^\top \mathbf{L}x = \|\mathbf{B}x\|^2 \geq 0$. That quantity is the discrete Dirichlet energy — the sum of squared differences across edges.
- $\mathbf{L}\mathbf{1} = 0$ always. Constants have zero gradient.

- The multiplicity of eigenvalue 0 is the number of connected components. Connectivity is a *spectral* property.
- The orientation choice cancels: flipping an edge negates a row of $\mathbf{B}$, which leaves $\mathbf{B}^T \mathbf{B}$ untouched.

Figure 2.1 works the construction through on a concrete graph.

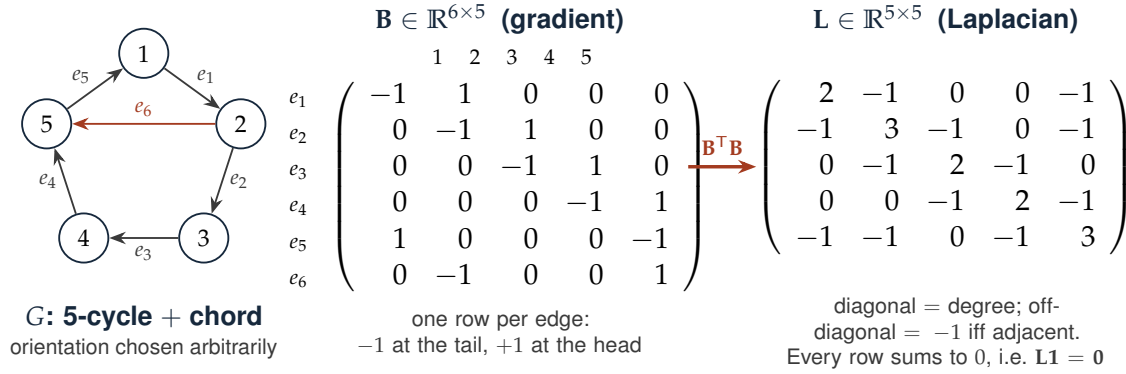


Figure 2.1. $\mathbf{L} = \mathbf{B}^T \mathbf{B} = \mathbf{D} - \mathbf{A}$, worked through. The incidence matrix is the discrete gradient, its transpose the discrete divergence, the composition the Laplacian. The $\mathbf{D} - \mathbf{A}$ form is what you would write down combinatorially; the $\mathbf{B}^T \mathbf{B}$ form is what tells you *why* the thing behaves like a second derivative. The spectrum of this $\mathbf{L}$ is $\{0, 1.382, 2.382, 3.618, 4.618\}$ — exactly one zero eigenvalue, confirming the graph is connected.

2.2 Now just write the PDEs down

With $\mathbf{L}$ in hand the classical equations transfer verbatim. Replace $-\nabla^2$ with $\mathbf{L}$, and u with a vector in $\mathbb{R}^n$:

Continuum	Graph	Solution
Heat $\partial_t u = \nabla^2 u$	$\dot{u} = -\mathbf{L}u$	$u(t) = e^{-\mathbf{L}t} u_0$
Wave $\partial_t^2 u = \nabla^2 u$	$\ddot{u} = -\mathbf{L}u$	$u(t) = \cos(\mathbf{L}^{1/2} t) u_0 + \dots$
Poisson $\nabla^2 u = f$	$\mathbf{L}u = f$	$u = \mathbf{L}^+ f$ ($\mathbf{L}$ is singular, so pseudo-inverse)
Schrödinger $i\partial_t \psi = -\nabla^2 \psi$	$i\dot{\psi} = \mathbf{L}\psi$	$\psi(t) = e^{-i\mathbf{L}t} \psi_0$

Figure 2.2 shows the heat equation actually integrated on a 16-cycle.

The gotcha — Deep message-passing GNNs oversmooth, and this is not an architectural defect that better engineering will remove. One round of message passing is one explicit Euler step of $\dot{u} = -\mathbf{L}u$; stacking k layers integrates the heat equation for time $\propto k$. The heat equation *converges to equilibrium* — that is what it is for. Node representations become indistinguishable because diffusion is working correctly. The mechanism hands you the mitigations, all of which are recognisable as numerical-PDE technique: residual connections (you are now integrating $\dot{u} = -\mathbf{L}u + u$, which has a non-trivial fixed point); reducing the effective Δt per layer; anisotropic or learned diffusion tensors; or running a *wave* equation instead, since $\ddot{u} = -\mathbf{L}u$ conserves energy

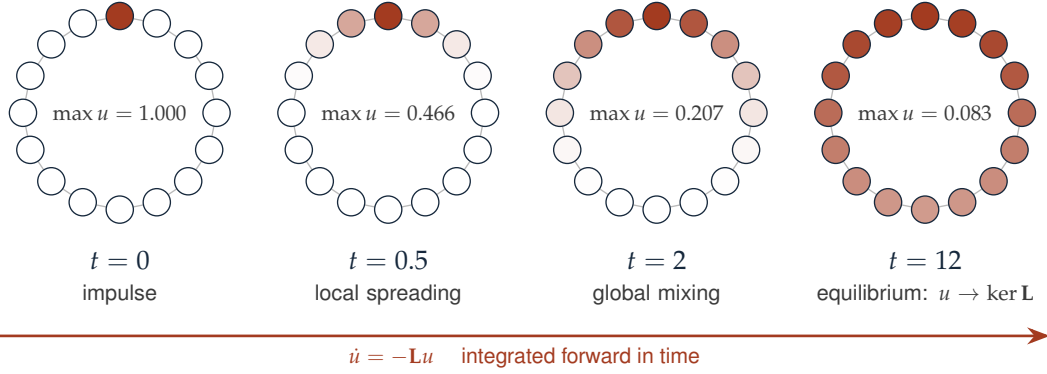


Figure 2.2. $u(t) = e^{-\mathbf{L}t}u_0$ on the 16-cycle, computed rather than sketched. An impulse at one vertex spreads, then homogenises. Node shading is normalised to the per-frame maximum; the printed figure is the true peak. Equilibrium for this graph is the uniform state $u_i = 1/16 = 0.0625$, and by $t = 12$ the peak has fallen to 0.0827 — most of the way there. **This figure is also the explanation of oversmoothing in deep graph neural networks:** the $t \rightarrow \infty$ limit is $\ker \mathbf{L}$, which on a connected graph is spanned by the constant vector. Every vertex ends with the same value.

and does not homogenise at all.

2.3 The correspondence runs both ways

This is the half that usually goes unremarked. It is not merely that graph problems can be phrased as PDEs. It is that *numerical PDE solving has always been graph computation*, and nobody was calling it that.

2.3.1 Discretisation produces a graph

Take any finite-difference or finite-element scheme. The stencil — the local pattern of neighbours each update touches — is an edge set. The assembled stiffness matrix is a weighted Laplacian. The sparsity pattern of that matrix is an adjacency matrix. Figure 2.3 shows the identification for the standard 5-point Laplacian.

2.3.2 Sampling a manifold produces a graph that converges to it

The other direction is deeper, and it is the foundation of manifold learning.

Take n points sampled from a d -dimensional manifold $\mathcal{M}$ embedded in $\mathbb{R}^D$. Connect each point to its k nearest neighbours, weighting edges by a Gaussian kernel $w_{ij} = \exp(-\|x_i - x_j\|^2/\varepsilon)$. Then as $n \rightarrow \infty$ and $\varepsilon \rightarrow 0$ at an appropriate joint rate, the suitably normalised graph Laplacian **converges to the Laplace–Beltrami operator on $\mathcal{M}$** (Belkin & Niyogi, 2005; Coifman & Lafon, 2006).

The graph does not merely *approximate* the manifold’s geometry. Its Laplacian converges to the manifold’s intrinsic differential operator. Geodesic structure, curvature and dimension are all recoverable from the spectrum and eigenfunctions of a graph built from nothing but nearest-neighbour distances.

This is what spectral clustering, diffusion maps and Laplacian eigenmaps actually do. Figure 2.4 shows the chain.

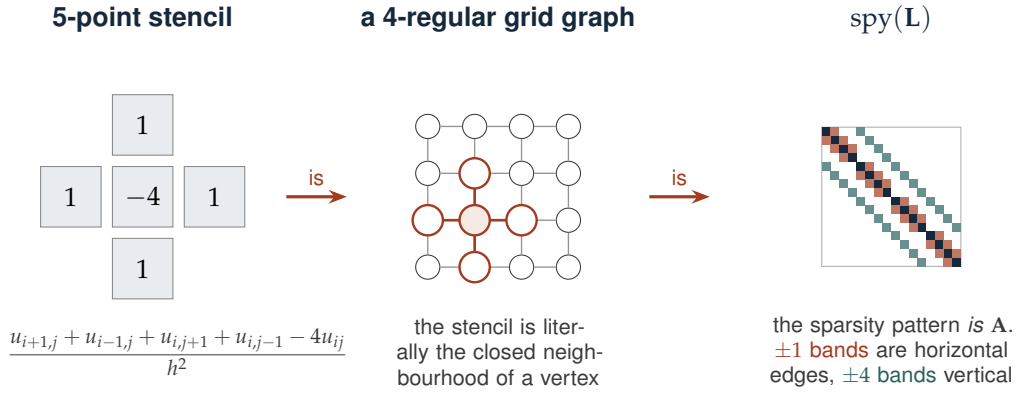


Figure 2.3. Three drawings of one object. A finite-difference stencil, the graph it induces, and the sparsity pattern of the assembled operator are the same information in three notations. Which is why the whole apparatus of sparse linear algebra is graph theory in practice: bandwidth minimisation is a graph labelling problem, nested dissection is recursive graph partitioning, and multigrid coarsening is graph coarsening.

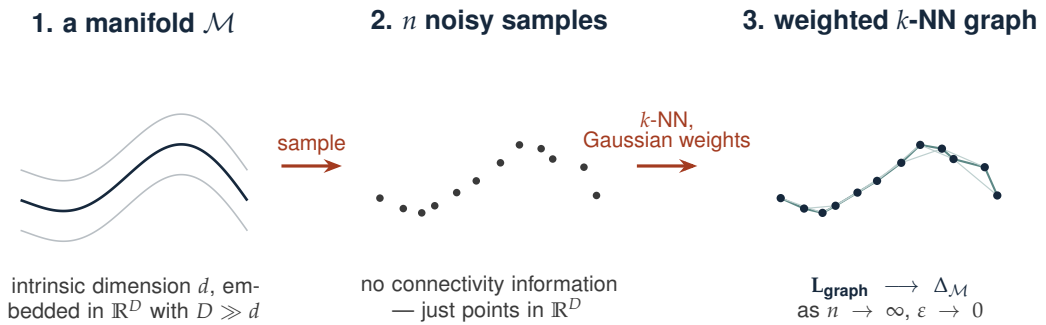


Figure 2.4. Belkin–Niyogi convergence. Sample a manifold, discard the geometry, keep only nearest-neighbour relations — and the graph Laplacian recovers the manifold’s intrinsic Laplace–Beltrami operator in the limit. The graph is not a crude stand-in for the manifold; it is a consistent estimator of its differential structure. The kernel bandwidth ε plays exactly the role that mesh size h plays in Figure 2.3: it is the same convergence question approached from opposite ends.

2.4 The payoff: GNNs are PDE solvers

Put the pieces together. A graph convolutional layer, stripped of notation, is

$$u^{(k+1)} = \sigma\left((\mathbf{I} - \Delta t \mathbf{L}) u^{(k)} \mathbf{W}\right)$$

and the bracket is one **explicit Euler step** of $\dot{u} = -\mathbf{L}u$. Message passing is time integration; layer depth is elapsed diffusion time.

Seen this way, the standard pathologies of deep GNNs stop being mysterious and become textbook numerical analysis:

GNN phenomenon	Numerical-analysis reading
Oversmoothing	Diffusion reaching equilibrium; $u \rightarrow \ker \mathbf{L}$
Depth capped at 2–4 layers	Total integration time before homogenisation
Residual connections help	They add a source term, moving the fixed point
Oversquashing at bottlenecks	Poor conditioning; small spectral gap across the cut
Graph rewiring helps	Preconditioning, or remeshing
Attention beats fixed weights	A learned anisotropic diffusion tensor

At the bench — The practical consequence is that the numerical-PDE literature is directly importable, and it is fifty years deeper than the GNN literature. When a deep graph model misbehaves, ask the questions you would ask of an unstable integrator: what is the effective Δt , is the scheme dissipative, what is the spectral radius of the update operator, would an implicit or higher-order scheme be better conditioned? GRAND (Chamberlain et al., 2021) is essentially the paper that asked those questions and replaced Euler with Runge–Kutta.

Glossary: The Laplacian: Graphs as Discrete Manifolds

Terms introduced in this chapter

Incidence matrix $\mathbf{B}$ — The $|E| \times |V|$ matrix with -1 at the tail and $+1$ at the head of each edge. It is the discrete exterior derivative: $\mathbf{B}f$ maps a function on vertices to its differences along edges. The discrete gradient.

Degree matrix $\mathbf{D}$ — Diagonal matrix with $D_{ii} = \deg(i)$.

Adjacency matrix $\mathbf{A}$ — $A_{ij} = 1$ (or w_{ij}) when $i \sim j$, else 0 .

Graph Laplacian $\mathbf{L}$ — $\mathbf{L} = \mathbf{B}^\top \mathbf{B} = \mathbf{D} - \mathbf{A}$. Composition of discrete divergence with discrete gradient, i.e. $-\nabla^2$. Positive semi-definite by construction, with $\mathbf{L}\mathbf{1} = 0$ always.

Discrete exterior calculus (DEC) — The framework giving a graph or simplicial complex a native differential calculus: k -forms as functions on k -simplices, with a coboundary operator playing the role of d . Graph calculus is the 0- and 1-form fragment.

0-form / 1-form — A 0-form is a function on vertices ($f : V \rightarrow \mathbb{R}$); a 1-form is a function on edges. $\mathbf{B}$ maps 0-forms to 1-forms.

Normalised Laplacian — $\mathbf{L}_{\text{sym}} = \mathbf{D}^{-1/2} \mathbf{L} \mathbf{D}^{-1/2}$, with spectrum confined to $[0, 2]$. Preferred when degrees are heterogeneous, since the unnormalised spectrum otherwise scales with the maximum degree.

Stiffness matrix — The sparse matrix assembled by a finite element discretisation. Its sparsity pattern is literally an adjacency matrix over the mesh; FEM assembly is graph construction.

Stencil — The local pattern of neighbours used by a finite-difference scheme (the 5-point Laplacian, say). The stencil defines the edge set of the induced grid graph.

Laplace–Beltrami operator — The generalisation of ∇^2 to a Riemannian manifold. Belkin–Niyogi: the graph Laplacian of a Gaussian-weighted k -NN graph on sampled points converges to it as $n \rightarrow \infty$ with bandwidth $\rightarrow 0$.

Diffusion map — Coifman–Lafon embedding using scaled Laplacian eigenvectors as coordinates, so that Euclidean distance in the embedding approximates diffusion distance on the manifold.

Heat kernel — $e^{-t\mathbf{L}}$, the solution operator of $\dot{u} = -\mathbf{L}u$. Its matrix entries give the amount of heat travelling between two vertices in time t .

Oversmoothing — The failure mode of deep message-passing networks: repeated application of a diffusion operator converges towards $\ker \mathbf{L}$, which on a connected graph is the constant function. Representations become indistinguishable. Not a bug in the architecture — it is the heat equation reaching equilibrium.

Explicit Euler — First-order time integration $u^{(k+1)} = u^{(k)} + \Delta t \cdot \dot{u}$. One round of graph message passing is exactly one explicit Euler step of graph diffusion.

CHAPTER 3

Fourier: The Eigenbasis of the Laplacian

Chapter 2 established that a graph has a Laplacian. This chapter is the observation that Fourier analysis was *always* a statement about that operator, and that the classical transform is one special case of it.

3.1 What the Fourier basis actually is

Start in the continuum. Why are complex exponentials the Fourier basis? Not by convention, and not merely because they are convenient. Because

$$\frac{d^2}{dx^2} e^{ikx} = -k^2 e^{ikx}$$

They are the **eigenfunctions of the Laplacian**, with eigenvalue $-k^2$. Everything else — the convolution theorem, Parseval, the diagonalisation of translation-invariant operators — follows from that.

“Frequency” is not a property of time or of space. It is an *eigenvalue of the Laplacian*. Any domain carrying a Laplacian therefore carries a Fourier transform — and by Chapter 2, a graph is such a domain.

So the generalisation writes itself. Eigendecompose the graph Laplacian,

$$\mathbf{L} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^T, \quad \mathbf{\Lambda} = \text{diag}(\lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n)$$

with $\mathbf{U}$ orthogonal because $\mathbf{L}$ is symmetric. Then define the **graph Fourier transform** and its inverse:

$$\hat{f} = \mathbf{U}^T f, \quad f = \mathbf{U} \hat{f}$$

Columns of $\mathbf{U}$ are the frequency basis; the λ_k are squared frequencies. The dictionary is exact:

Classical	Graph	Note
e^{ikx}	column u_k of $\mathbf{U}$	the eigenvector
k^2	λ_k	the eigenvalue
$\hat{f}(k) = \int f e^{-ikx}$	$\hat{f} = \mathbf{U}^T f$	projection onto the basis
low frequency = smooth	small λ = smooth on the graph	since $u^T \mathbf{L} u$ is Dirichlet energy
$\widehat{f * g} = \hat{f} \cdot \hat{g}$	$g(\mathbf{L})f = \mathbf{U} g(\mathbf{\Lambda}) \mathbf{U}^T f$	holds <i>by definition</i> on graphs

That last row deserves emphasis. On a general graph there is no translation operator, so there is no way to define convolution as “slide one signal past the other”. Instead graph convolution is *defined* as multiplication in the spectral domain. The convolution theorem is not a theorem here — it is the definition, chosen precisely so that the classical case is recovered.

3.2 The check: the DFT is the GFT on a ring

This is not hand-waving, and you can verify it in a few lines.

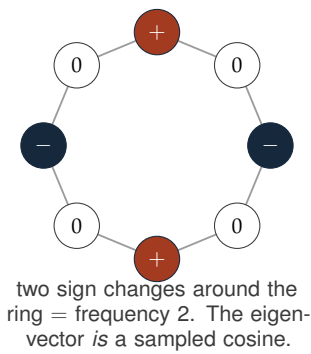
Take the cycle graph C_n . Its Laplacian is **circulant** — every row a cyclic shift of the last — because the graph is vertex-transitive under rotation. Every circulant matrix is diagonalised by the DFT basis. Hence:

The eigenvectors of L_{C_n} are exactly the DFT basis vectors $u_k(j) = \frac{1}{\sqrt{n}} e^{2\pi i j k / n}$, with eigenvalues

$$\lambda_k = 2 - 2 \cos\left(\frac{2\pi k}{n}\right) \approx \left(\frac{2\pi k}{n}\right)^2 \quad \text{for small } k.$$

The classical DFT is the graph Fourier transform on a ring. The approximation on the right is the discrete spectrum converging to the continuum one, k^2 , exactly as in Chapter 2.

C_8 , with the mode u_2 marked



computed spectrum of $L_{C_{32}}$

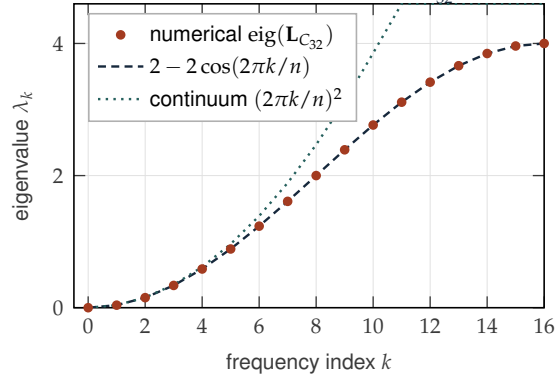


Figure 3.1. The DFT is the GFT on a ring. Left: the eigenvector u_2 of C_8 — a sampled cosine with two sign changes. Frequency, read straight off the graph. Right: eigenvalues of $L_{C_{32}}$ computed numerically (points) against the closed form $2 - 2 \cos(2\pi k/n)$ (dashed); they agree to 1.8×10^{-15} , i.e. machine precision. The dotted continuum curve $(2\pi k/n)^2$ tracks closely at low k (0.03843 against 0.03855 at $k = 1$) and departs at high k — the discrete spectrum saturating, because a graph has a maximum frequency, bounded by $\lambda_{\max} \leq 2d_{\max}$.

Sanity check — Six lines of numpy settles it: build $L = 2 \cdot \text{eye}(n) - \text{roll}(\text{eye}(n), 1, 0) - \text{roll}(\text{eye}(n), -1, 0)$, then compare $\text{sort}(\text{eigvalsh}(L))$ against the sorted closed-form values.

Watch for the degeneracy: on a ring, every non-zero eigenvalue has multiplicity two — k and $n - k$ give the same λ , being the sine and cosine at that frequency. A naive index-by-index comparison against $k = 0, 1, 2, \dots$ will report a maximum error around 2.2 and look like a refutation. It is not; it is the multiplicity. This was a real error in

the first draft of Figure 3.1, caught only because the number was printed rather than assumed.

The same argument on a *path* graph — a ring cut open, which is to say Neumann boundary conditions — gives the DCT basis instead. Figure 3.2 shows the first four modes.

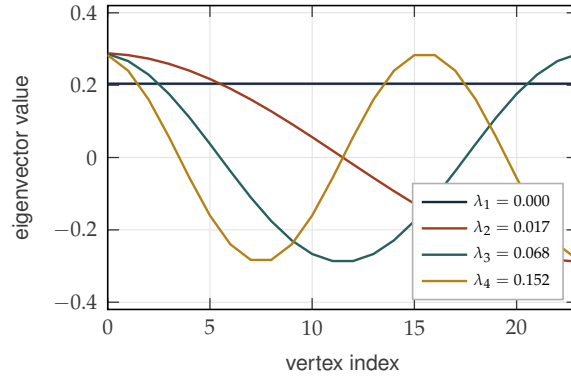


Figure 3.2. The first four eigenvectors of a 24-vertex path graph. These are half-cosines — the DCT-II basis — arrived at with no reference to trigonometry whatsoever, purely by eigendecomposing $\mathbf{D} - \mathbf{A}$. The eigenvalues $\lambda_k = 4 \sin^2(\pi k / 2n)$ increase with the number of sign changes, which is the graph-theoretic definition of frequency. Hold this figure for Chapter 4: sinusoidal positional encodings in a transformer are *these vectors*, because a token sequence is a path graph.

3.3 Why there is no fast graph Fourier transform

The classical FFT runs in $O(n \log n)$. The graph Fourier transform needs a full eigendecomposition at $O(n^3)$, then $O(n^2)$ per application. This is not an engineering gap waiting to be closed; the reason is structural.

Cooley–Tukey works by *recursively exploiting the subgroup lattice of $\mathbb{Z}/n\mathbb{Z}$* . Splitting into even and odd indices is a coset decomposition; the recursion is over subgroups. The algorithm’s precondition is that the domain be a group.

A ring is a group. A general graph is not. It has no translation operator, no subgroups, and so nothing to recurse on.

The FFT is fast because of **symmetry**, not because of sparsity. Remove the group structure and the speed goes with it. This one fact drives the entire design of practical spectral graph neural networks.

3.3.1 The workaround: polynomials in $\mathbf{L}$

If you cannot diagonalise, approximate the filter by a polynomial:

$$g(\mathbf{L}) \approx \sum_{k=0}^K \theta_k T_k(\mathbf{L})$$

with T_k the Chebyshev polynomials, chosen for numerical stability and their minimax approximation property on the spectral interval.

Two things then happen at once, and the coincidence is the entire point:

1. **No eigendecomposition is needed.** Evaluating a polynomial in $\mathbf{L}$ costs K sparse matrix–vector products: $O(K|E|)$.
2. **The filter becomes spatially localised.** $(\mathbf{L}^k)_{ij} \neq 0$ only when $\text{dist}(i, j) \leq k$. A degree- K polynomial filter is therefore *exactly* K -hop local.

Polynomial order and spatial locality are the same number (Figure 3.3). Which is why a GCN layer with $K = 1$ aggregates over immediate neighbours only: it is a first-order spectral filter and nothing else.

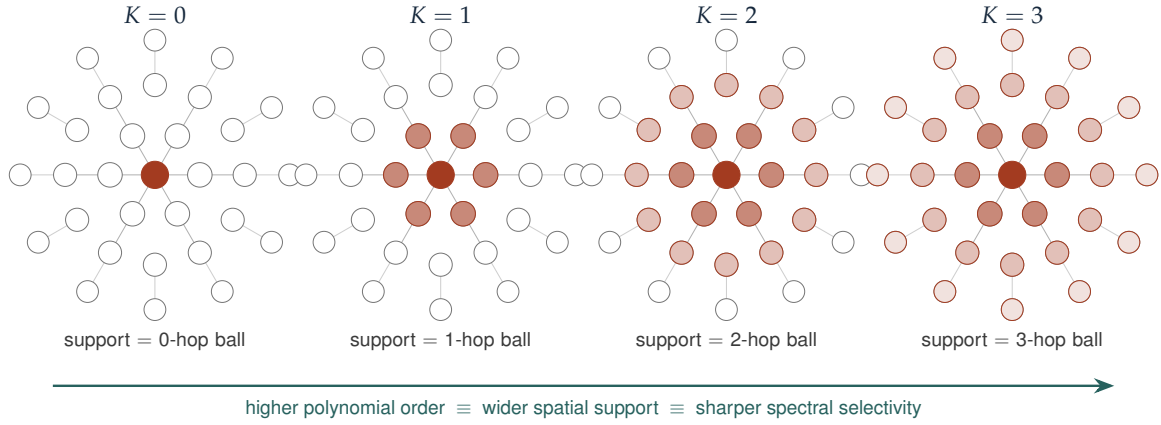


Figure 3.3. Polynomial order *is* spatial locality. Because $(\mathbf{L}^k)_{ij}$ vanishes whenever i and j are more than k hops apart, a degree- K filter reaches exactly the K -hop ball. Raising K buys sharper spectral selectivity and pays in support — which is the graph statement of the classical time–frequency uncertainty principle. A spectrally sharp filter cannot be spatially compact, on a graph for the same reason as on the line.

3.4 The unification

Chapters 2 and 3 converge on a single operator, which is worth drawing explicitly (Figure 3.4).

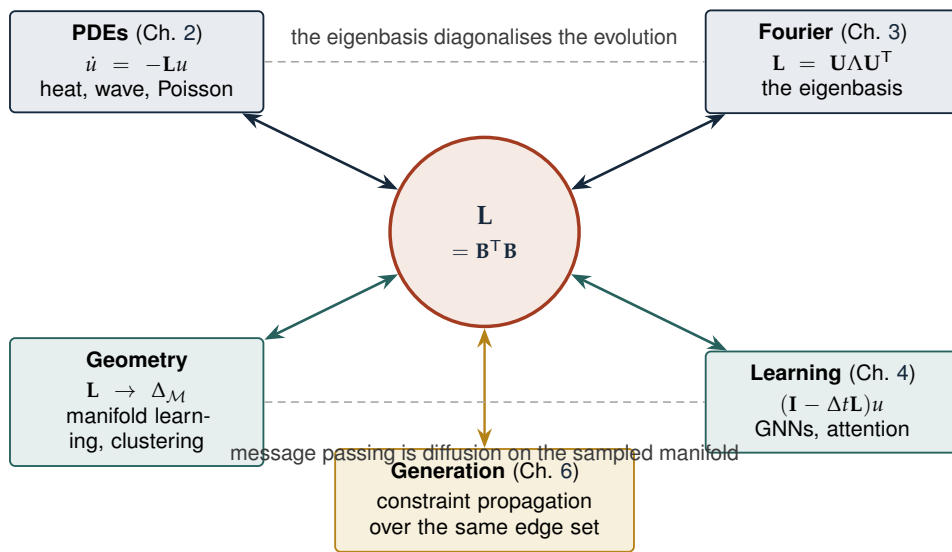


Figure 3.4. One operator, five literatures. Each box is a field with its own vocabulary, conferences and textbooks; each is a view of $B^T B$. The dashed links are cross-connections that only become visible once the centre is named — that spectral clustering and low-pass graph filtering are the same computation, for instance, or that a GNN’s practical depth limit and a diffusion timescale are the same quantity.

Glossary: Fourier: The Eigenbasis of the Laplacian

Terms introduced in this chapter

Eigenbasis — A basis of eigenvectors of an operator. Diagonalising the operator in this basis is the entire content of ‘working in the frequency domain’.

Graph Fourier transform (GFT) — Eigendecompose $\mathbf{L} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$; then $\hat{f} = \mathbf{U}^\top f$. Columns of $\mathbf{U}$ are the frequency basis, λ_k are squared frequencies.

Circulant matrix — A matrix in which each row is a cyclic shift of the previous one. Every circulant is diagonalised by the DFT basis — the algebraic reason the classical Fourier transform is the graph Fourier transform on a ring.

Cooley–Tukey — The $O(n \log n)$ FFT, which recursively exploits the subgroup lattice of $\mathbb{Z}/n\mathbb{Z}$. Its prerequisite is group structure, which a general graph does not have; hence no fast GFT and an $O(n^3)$ eigendecomposition.

Spectral filter — A function $g(\lambda)$ applied to the eigenvalues: $g(\mathbf{L})f = \mathbf{U}g(\mathbf{\Lambda})\mathbf{U}^\top f$. Graph convolution is *defined* this way, so the convolution theorem holds by construction.

Chebyshev approximation — Approximating $g(\lambda)$ by a degree- K polynomial in $\mathbf{L}$. Because $\mathbf{L}^K$ has support exactly on the K -hop neighbourhood, polynomial order equals spatial locality — and the eigendecomposition is avoided entirely (ChebNet, Defferrard 2016).

Algebraic connectivity (Fiedler value) — λ_2 , the smallest non-zero Laplacian eigenvalue. Zero iff the graph is disconnected; its eigenvector (the Fiedler vector) gives the standard spectral bisection.

Spectral gap — $\lambda_2 - \lambda_1$. Controls mixing time of random walks and the convergence rate of diffusion; large gap means fast homogenisation, hence rapid oversmoothing.

Uncertainty principle — A signal cannot be arbitrarily concentrated in both vertex and spectral domains. The reason a sharp spectral filter is spatially delocalised and a K -hop filter cannot be spectrally sharp.

Band-limited signal — A signal supported on the low- λ eigenvectors, i.e. smooth with respect to the graph. The assumption underlying graph sampling theory and most GNN inductive bias.

CHAPTER 4

Language Models: What the Data Actually Is

Are all LLMs made from paired JSON data? No. The question contains a category error worth unpicking carefully, because the mistake it encodes — confusing a *storage container* with a *training objective* — is the same mistake that makes the rest of the architecture opaque.

Then, having cleared that up, we come back to the Laplacian, because attention is a learned adjacency matrix and a transformer is a graph neural network on a complete digraph.

4.1 Three stages, three different data shapes

There is no single answer to “what is the training data”, because there are three distinct stages with genuinely different structure (Figure 4.1).

4.1.1 Pretraining: no pairs at all

Raw unstructured text — web crawl, books, code, papers. It is tokenised into a flat stream of integers by a learned subword vocabulary (BPE or unigram), and the objective is

$$\max_{\theta} \sum_t \log P_{\theta}(x_t \mid x_{<t})$$

That is the whole thing. There is no input, no output, no pair. Every token is simultaneously a target (predicted from its predecessors) and context (for its successors). One document of length T supplies T training signals, not one.

JSONL appears at this stage, and this is where the misconception starts. Corpora are commonly sharded as newline-delimited JSON: `{"text": "..."} per line`. But that is a **container** — a convenience for streaming and deduplication. The payload is one plain text field. The model never sees the braces. Change the format to Parquet, WebDataset or raw `.txt` shards and nothing about the training changes.

Format is orthogonal to objective. JSONL is how bytes are arranged on disk; next-token prediction is what happens to them afterwards. The presence of JSON in a pipeline says nothing whatsoever about whether the learning is paired.

4.1.2 SFT: paired-looking, but flattened

Supervised fine-tuning is where the intuition behind “paired JSON” has real purchase — and where it is still not quite right.

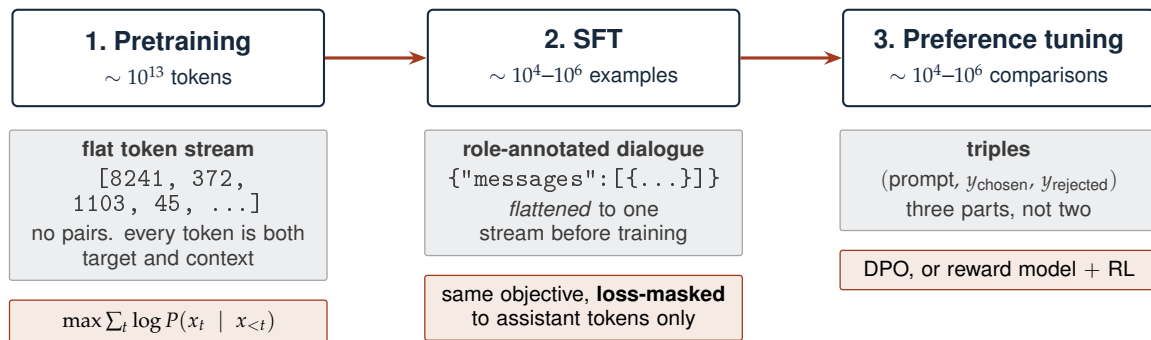
The data genuinely is structured, conventionally as `{"messages": [{"role": "user", ...}, {"role": "assistant", ...}]}`. But before a single gradient step, that structure is

flattened into one token stream with special role-delimiter tokens marking the boundaries. The objective remains $\log P(x_t | x_{<t})$, unchanged from pretraining.

What makes it behave like supervised learning is not the JSON. It is **loss masking**: the per-token loss is multiplied by zero on prompt and user tokens, and by one on assistant tokens. Gradients flow only from what the model was supposed to say (Figure 4.2).

4.1.3 Preference tuning: triples, not pairs

The third stage uses (prompt, y_{chosen} , y_{rejected}) — a triple. DPO optimises directly on these; classical RLHF fits a reward model to them and then runs policy optimisation against it. Either way the unit of data has three parts, so “pairs” is wrong here in a second, different sense.



Where “paired JSON” comes from: only stage 2 is stored as structured JSON, and even there the structure is discarded before training. Stage 1 is unstructured text that merely *ships* in JSONL containers. Stage 3 is triples. At no point is the model trained on JSON — it is trained on integers.

Figure 4.1. Three stages, three shapes. The intuition that LLMs learn from input/output pairs is a reasonable extrapolation from ordinary supervised learning, and it is wrong in three separate ways: pretraining has no pairs, SFT’s pairs are flattened away before training, and preference tuning uses triples. What is common across all three is next-token prediction; what differs is which tokens the loss is allowed to see.

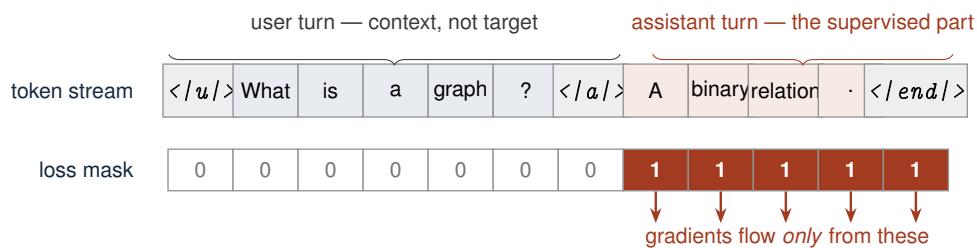


Figure 4.2. Loss masking is what makes SFT “supervised”. The conversation has been flattened into a single sequence with role-delimiter tokens; the objective is still $\log P(x_t | x_{<t})$ at every position. Multiplying the per-token loss by a 0/1 mask is the entire mechanism that converts next-token prediction into instruction following. Without the mask the model would be trained to generate plausible *user* turns as well — which is occasionally what you want, and is exactly how you would build a user simulator.

4.2 Attention is a learned adjacency matrix

Now the connection back to Chapter 2. Write a single attention head:

$$\text{Attn}(X) = \underbrace{\text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)}_{\mathbf{A} \in \mathbb{R}^{n \times n}} V$$

The bracketed factor is an $n \times n$ non-negative matrix whose rows sum to one. That is a **row-stochastic weighted adjacency matrix** over tokens. The multiplication by V is aggregation of neighbour features. So:

A transformer layer is a graph neural network on a *complete weighted digraph* over tokens, where the adjacency matrix is recomputed from the data at every layer and in every head. Causal masking prunes that complete digraph to a DAG. “Attention” and “learned dynamic adjacency” are the same object under two names.

Figure 4.3 draws the identification.

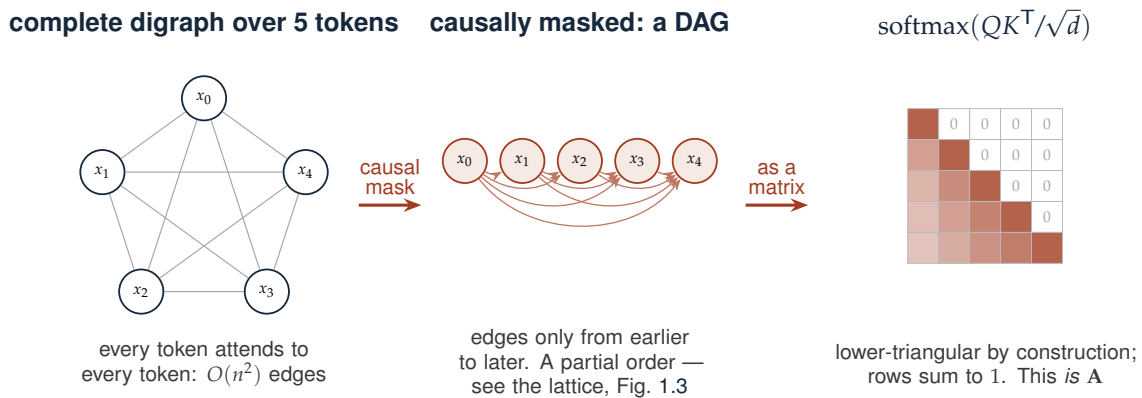


Figure 4.3. Three views of one operation. A complete digraph over tokens, pruned by the causal mask to a DAG, stored as a lower-triangular row-stochastic matrix. Reading attention as adjacency explains several things at once: why cost is $O(n^2)$ (a complete graph has $O(n^2)$ edges); why sparse-attention variants work (they are edge-sampling schemes on that graph — local windows, strided patterns, learned sparsity); and why multi-head attention helps (each head is a separate relation type, exactly as in a multi-relational property graph).

At the bench — This reframing is not merely aesthetic — it makes the sparse-attention literature legible as a single design space. Longformer’s sliding window is a path-graph neighbourhood; BigBird adds random edges plus global hub nodes, which is a small-world construction chosen to preserve the spectral gap; and linear attention factorises the adjacency into a low-rank product. Every one of them is answering the same question: *which edges of the complete graph can I delete without wrecking the spectrum?*

4.3 Positional encoding closes the loop

There is one more identification, and it is the neatest in the document.

A transformer is permutation-equivariant — attention has no notion of order. Position must be injected. The original construction used sinusoids of geometrically spaced frequencies.

Why sinusoids? Chapter 3 already answered this. **A token sequence is a path graph.** Sinusoids are the Laplacian eigenvectors of a path graph (Figure 3.2). So sinusoidal positional encoding is *spectral encoding of the sequence graph* — writing each position in the Fourier basis of the domain it lives on.

Which immediately tells you how to generalise. For a transformer over an arbitrary graph, the correct positional encoding is not a sinusoid: it is the **Laplacian eigenvectors of that graph**. This is exactly what graph transformers do, and it was derived rather than guessed.

Domain	Its graph	Correct positional encoding
Text sequence	path graph	sinusoids = path-graph eigenvectors
Image patches	grid graph	2D sinusoids = grid eigenvectors
Molecule	molecular graph	Laplacian eigenvectors of that graph
Arbitrary graph	itself	Laplacian eigenvectors; sign ambiguity must be handled

The gotcha — Laplacian eigenvector positional encodings carry a **sign and basis ambiguity** that sinusoids do not, and it is a real obstacle rather than a technicality. If u_k is an eigenvector then so is $-u_k$, with the same eigenvalue; and where an eigenvalue is degenerate — which happens constantly on symmetric graphs, as it did on the ring in Figure 3.1 — *any orthonormal basis of that eigenspace* is equally valid. Train naively and the model overfits to whichever arbitrary signs and basis vectors `eigh` happened to return, then fails on isomorphic graphs.

Mitigations, in increasing order of cost: random sign flipping during training; sign-invariant architectures (SignNet, BasisNet) which are equivariant to the ambiguity by construction; or sidestepping eigenvectors entirely in favour of random-walk encodings, which use the diagonal of $\mathbf{P}^k$ and are invariant because they never pick a basis.

Glossary: Language Models: What the Data Actually Is

Terms introduced in this chapter

Tokenisation — Mapping text to integers via a learned subword vocabulary (BPE, unigram). The model never sees characters, let alone file formats.

Autoregressive objective — Maximise $\sum_t \log P(x_t \mid x_{<t})$. Pretraining is this and nothing else; there are no input/output pairs at this stage.

JSONL — Newline-delimited JSON. A *storage container* for shards, not a training structure — the payload is typically one plain text field. Confusing the container with the objective is the origin of the ‘paired JSON’ misconception.

Supervised fine-tuning (SFT) — Training on role-annotated conversations. Superficially paired, but flattened to a single token stream with role-delimiter tokens before training.

Loss masking — Zeroing the loss on prompt/user tokens so gradients flow only from assistant tokens. This — not the JSON — is what makes SFT behave like supervised learning while remaining next-token prediction underneath.

Preference triple — (prompt, y_{chosen} , y_{rejected}). The unit of DPO and reward model training. Triples, not pairs.

DPO — Direct Preference Optimisation: a closed-form reparameterisation that optimises the RLHF objective directly on preference triples, with no explicit reward model or rollouts.

Attention as adjacency — $\text{softmax}(QK^T / \sqrt{d})$ is a row-stochastic weighted adjacency matrix over tokens, recomputed per layer and per head. A transformer layer is a GNN on a complete digraph.

Causal mask — Setting attention logits to $-\infty$ above the diagonal, which prunes the complete digraph to a DAG respecting sequence order.

Positional encoding — Injected position information. Sinusoidal PEs are the Fourier basis of a path graph; the topology-general version is Laplacian eigenvector PE, used by graph transformers.

Context window — The maximum token count attended over. Bounds the diameter of the token graph, and drives the quadratic $O(n^2)$ cost of a complete adjacency.

CHAPTER 5

Encoding Script: Kufic and Blackletter

Square Kufic and blackletter look superficially similar as problems — both are historical scripts with strong internal rules, both seem like obvious candidates for procedural generation. They are in fact *opposite* problems and want opposite machinery. Getting this wrong is the main way the project fails, so it is worth separating them before writing any code.

	Square Kufic	Blackletter
Substrate	discrete grid	continuous plane
Primitive	the module (unit cell)	the pen stroke
Rules are	combinatorial constraints	geometric constraints
Natural model	CSP on a grid graph	skeleton graph + sweep operator
Solver	clingo, SAT, WFC with backtracking	sparse linear solve (METAFONT)
Hard part	<i>global</i> connectivity	attachment continuity

5.1 Square Kufic: a discrete CSP on a grid graph

Square Kufic (*banna'i*, “builder’s”) is the architectural brick-lattice style found across Persian and Central Asian tilework. Its defining rule is brutally simple and completely determines the aesthetic:

Stroke width = gap width = one module. Everything else follows. All strokes are axis-aligned, all features are quantised to the grid, and the letterforms are whatever survives that constraint.

This is a near-perfect fit for constraint satisfaction, and it decomposes into two layers that should be solved separately.

5.1.1 Layer A: the orthographic FSA

Arabic is a joining script. Each letter takes one of four positional forms — isolated, initial, medial, final — determined entirely by its immediate neighbours and by whether those neighbours are joining letters. A small set of letters (*alif*, *dal*, *dhal*, *ra*, *zay*, *waw* and relatives) never connect to what follows, forcing a break.

Because the rule is purely local, it is a deterministic finite automaton — which is to say, a labelled directed graph (Figure 5.1). This layer is easy, and it is worth implementing first and separately, because it is the part that can be exhaustively tested.

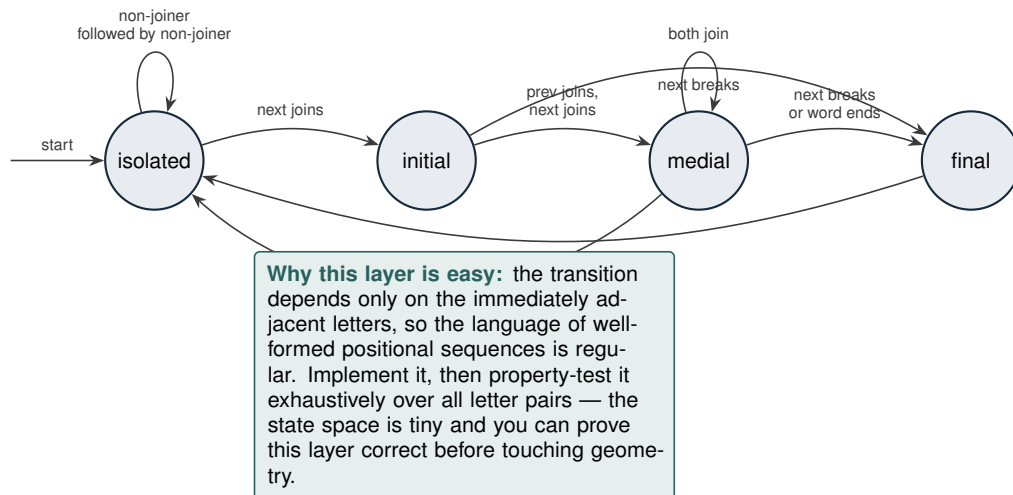


Figure 5.1. Arabic joining behaviour is a DFA, hence a labelled digraph. States are positional forms; transitions are determined by neighbour joining-class. The non-joining letters are the source of essentially all the branching: they end a connected run mid-word, which is why Arabic words are made of one or more *connected components* rather than one continuous stroke — a fact that matters enormously for the geometric layer below.

5.1.2 Layer B: the geometric CSP

Now the interesting part. The canvas is a grid graph with 4-adjacency; each cell is a variable with domain {ink, void}. The constraints are:

1. stroke width = gap width = 1 module (the style rule);
2. no 2×2 solid block — these read as blobs, not strokes;
3. minimum separation between distinct letters, so they remain individually legible;
4. **each letter's ink cells form one connected component.**

Composition is then: embed the letter sequence into the grid subject to all of the above. That is a constrained self-avoiding walk / graph embedding problem, and it is NP-hard in general.

Figure 5.2 shows a valid patch and the two characteristic failures.

The gotcha — Connectivity is not expressible as local arc consistency, and this is where naive WFC dies. Run a standard tile-based generator over a Kufic constraint set and it will happily produce beautiful, rule-abiding, completely disconnected letter shrapnel — indefinitely, with no error, because from the propagator's point of view nothing is wrong.

The mechanism: arc consistency reasons about pairs of adjacent variables. Connectivity is a property of the entire assignment. No conjunction of binary constraints implies it. Three fixes that actually work, in increasing order of effort:

1. **Grow from a seed.** Generate by expanding an already-connected region rather than by filling cells independently. Connectivity holds by construction. Cheapest, least flexible.
2. **Encode reachability explicitly.** In SAT, add single-source flow variables or reachability-level variables: each ink cell must have a neighbour at a strictly

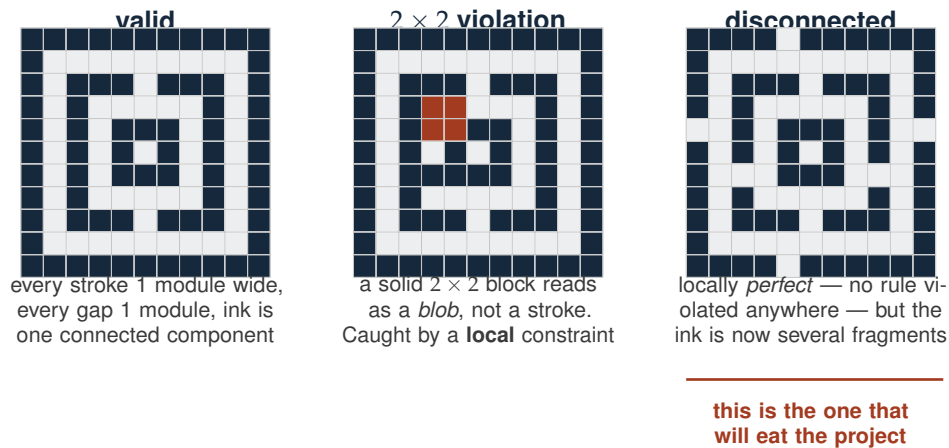


Figure 5.2. Two failures, two completely different difficulties. The 2×2 blob is a *local* constraint: it involves four mutually adjacent cells, so arc consistency catches it and a naive propagator will never produce one. Disconnection is a *global* constraint — every local neighbourhood in the right-hand patch is individually legal, and no amount of local propagation can detect the problem. These patches are schematic lattice motifs demonstrating the constraint system, not transcriptions of specific words.

lower level, with one seed at level 0. Costs $O(|V| \log |V|)$ extra variables and works.

3. **Use a solver that has it natively.** ASP (clingo) supports reachability directly and is, for this reason, the best-fit tool for square Kufic. This is the option to reach for first.

Budget for this explicitly. It is not a detail to be patched at the end — it determines which solver you should have chosen at the start.

5.2 Blackletter: continuous constraints, and why gridding it is wrong

Textura and Fraktur are **pen-generated**. A broad nib is held at a fixed angle — roughly 30° to 40° — and swept along a path. The letterform is the *envelope* of that sweep, not a set of pixels. Stroke weight is not a parameter you choose; it is a consequence of the angle between the nib and the direction of travel (Figure 5.3).

5.2.1 The right structure: a part graph

This is Knuth’s METAFONT model, and blackletter is the script where it is most faithful precisely because the pen mechanics fully determine the form.

- **Nodes** are reusable parts: stem, diamond terminal, foot, hairline connector, arch.
- **Edges** are attachment relations, carrying geometric constraints (this foot meets the baseline at 45° ; this arch springs from the stem at $0.62 \times$ x-height).
- **Letters** are instantiations of a small, heavily reused part library.

Textura’s notorious illegibility — the reason *minim* sequences in mediaeval manuscripts are genuinely ambiguous — is direct evidence that the part graph is small and the same

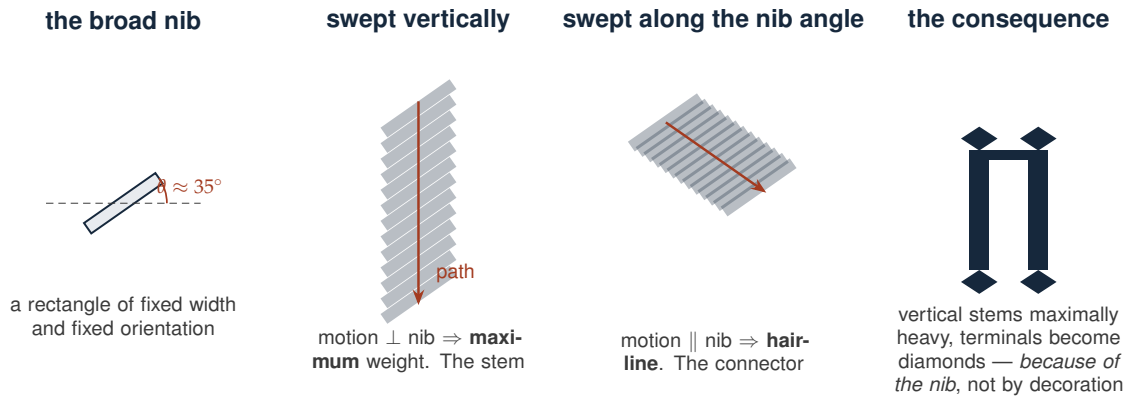


Figure 5.3. Blackletter is a consequence of a tool, not a style applied on top of one. Because stroke weight varies as the cosine of the angle between travel direction and nib angle, vertical strokes come out maximally heavy and diagonal connectors come out as hairlines. The characteristic diamond terminals are simply what the end of a rectangular nib leaves behind. Model the nib and the style is implied; model the outlines directly and you will spend forever hand-tuning shapes that would have fallen out for free.

stem repeats everywhere. That is a structural fact about the script, and a good procedural model should reproduce it rather than fight it (Figure 5.4).

Solving the attachment constraints means solving a sparse linear system over point variables. Which is, once again, a sparse solve on a graph — Chapter 2, in a different costume.

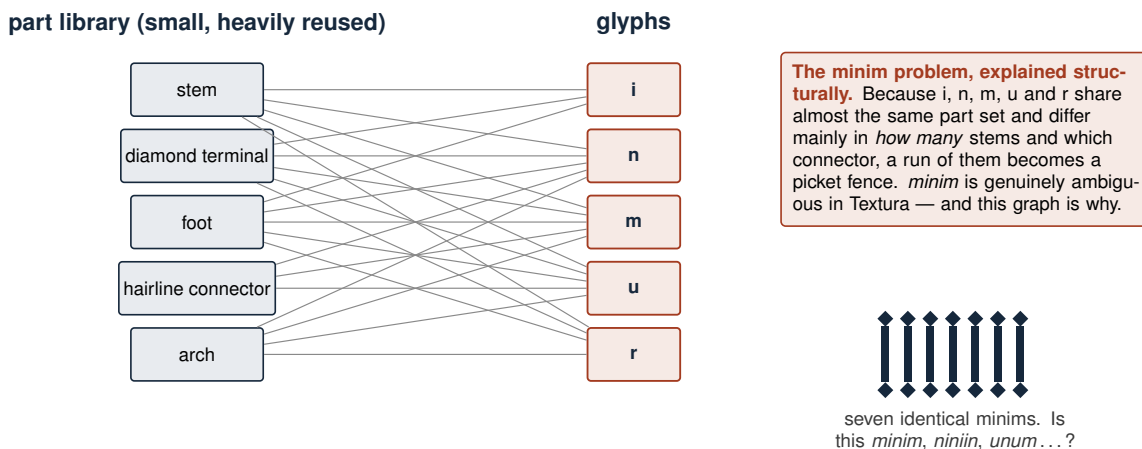


Figure 5.4. A bipartite part-glyph graph. Blackletter's aesthetic unity and its illegibility are the same structural fact: a very small part library, reused with near-total regularity. A generative model built on this graph reproduces both properties automatically. Note this is the reverse of the Kufic situation — here the difficulty is not global connectivity but *local attachment continuity*, keeping tangents and weights consistent where parts meet.

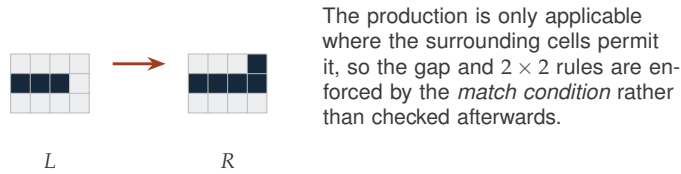
5.3 The unifying move: don't encode rules *in* a graph

The original question was how to encode the rules of a script *into* a graph. The better move is to encode them **as rewrites on graphs**.

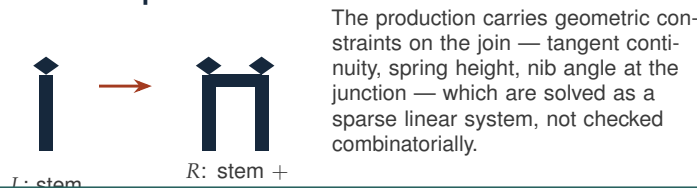
Graph grammars — and their geometric cousin, Stiny's shape grammars — are productions $L \rightarrow R$: find a subgraph matching L , replace it with R . The grammar is the ruleset;

the graph is the workpiece. Both scripts fit (Figure 5.5), and so, as it happens, do L-systems and dungeon generation — see Chapter 6.

Kufic production: extend a stroke, preserving the gap rule



Blackletter production: attach an arch to a stem



Same formalism, different constraint algebra. In both cases a production matches a subgraph and rewrites it. What differs is the *kind* of constraint attached to the match: Kufic's are combinatorial (over $\{\text{ink}, \text{void}\}$) and Blackletter's are geometric (over real-valued point positions). This is why one wants a SAT/ASP solver and the other wants a linear solve — and why trying to force either into the other's tooling is the characteristic way these projects stall.

Figure 5.5. Encode the rules *as* rewrites, not *in* a data structure. A graph grammar separates the ruleset (productions) from the workpiece (the graph), which means the rules stay inspectable, composable and individually testable. This is the formalism to reach for, and it is the same one that turns up in Chapter 6 as L-systems and mission-graph generation.

Glossary: Encoding Script: Kufic and Blackletter as Graphs

Terms introduced in this chapter

Finite state automaton (FSA) — A labelled directed graph with designated start and accepting states. Arabic contextual joining is a deterministic FSA over four positional forms.

Joining form — One of *isolated*, *initial*, *medial*, *final*. Which form a letter takes is determined by its neighbours and by whether each is a joining letter — purely local, hence expressible as an FSA transition.

Non-joining letter — The set alif, dal, dhal, ra, zay, waw (alif, dal, dhal, ra, zay, waw and relatives) which never connect to the following letter. They force a break in the connected run — the primary source of the FSA's branching.

Banna'i / square Kufic — The architectural brick-lattice style in which stroke width and gap width are equal to one module and all strokes are axis-aligned on a grid.

Module — The unit cell of the Kufic grid. Style is defined by the constraint that ink width and void width are both exactly one module.

Constraint satisfaction problem (CSP) — Variables with finite domains, plus constraints restricting joint assignments. The Kufic layout problem is a CSP on a grid graph with domain {ink, void}.

Arc consistency — The condition that every value in a variable's domain has a supporting value in each neighbour's domain. Enforced by AC-3. Purely *local* — which is why it cannot see connectivity.

Connectivity constraint — The requirement that a letter's ink cells form one connected component. A *global* constraint: no amount of local propagation implies it. Encoded in SAT via single-source flow or reachability levels.

Self-avoiding walk — A path in a graph that revisits no vertex. Embedding a letter sequence into the Kufic grid subject to spacing constraints is a constrained self-avoiding walk problem, NP-hard in general.

Answer set programming (ASP) — Declarative logic programming over stable models, with `clingo` the standard solver. Strong native support for reachability and cardinality constraints, which makes it the best-fit solver for square Kufic.

Broad-nib model — The generative model of blackletter: a rectangular pen held at fixed angle ($\approx 30\text{--}40^\circ$) and swept along a path. Stroke weight is a *consequence* of direction, not a parameter.

Skeleton and envelope — The skeleton is the path the pen travels; the envelope is the outline swept out. Blackletter is naturally authored as skeleton + nib, and only rasterised at the end.

METAFONT model — Knuth's parametric system in which glyphs are pen paths with constraints solved as a sparse linear system over point variables. Blackletter is the script where this model is most faithful, because the pen mechanics fully determine the form.

Part graph — A graph whose nodes are reusable glyph components (stem, diamond terminal, foot, hairline) and whose edges are attachment constraints. Textura's illegibility is direct evidence that its part graph is small and heavily reused.

Graph grammar / shape grammar — A rewriting system of productions $L \rightarrow R$ that match a subgraph and replace it (Stiny, for shapes). The right formalism for encoding script rules: the grammar is the ruleset, the graph is the workpiece.

CHAPTER 6

Wave Function Collapse as Constraint Solving

Does wave function collapse use graphs? Natively — and more than that, WFC is a graph algorithm wearing borrowed physics vocabulary. Stripping the vocabulary away is the single most useful thing you can do to it, because it connects the algorithm to fifty years of constraint-programming literature and tells you exactly what its failure modes are and why.

6.1 The translation

Maxim Gumin’s algorithm, in its own terms: each cell holds a *superposition* of possible tiles; repeatedly *collapse* the lowest-*entropy* cell to a single tile, then *propagate* the consequences to neighbours; stop when everything is collapsed or a *contradiction* occurs.

Now the same algorithm in constraint-satisfaction terms:

WFC vocabulary	CSP vocabulary	Since about
cell	variable	1974
superposition	domain	1974
tile adjacency rules	binary constraints	1974
collapse	assignment	—
propagation	arc consistency (AC-3)	Mackworth, 1977
minimum entropy heuristic	minimum remaining values (MRV)	Haralick & Elliott, 1980
contradiction	domain wipeout	—
restart on contradiction	(an unusually weak response)	—

WFC is **AC-3 with a most-constrained-variable heuristic and weighted random assignment**. There is no physics in it. The name is a metaphor that has cost the technique a great deal of transferable theory — once you see the mapping, the entire constraint-programming toolbox becomes available.

Figure 6.1 traces one collapse-and-propagate cycle.

6.2 Why contradictions happen, and why restarting is a choice

Arc consistency reasons about *pairs* of variables. It guarantees that every remaining value has a supporting partner in each neighbour — and nothing more. It does not guarantee that a globally consistent completion exists.

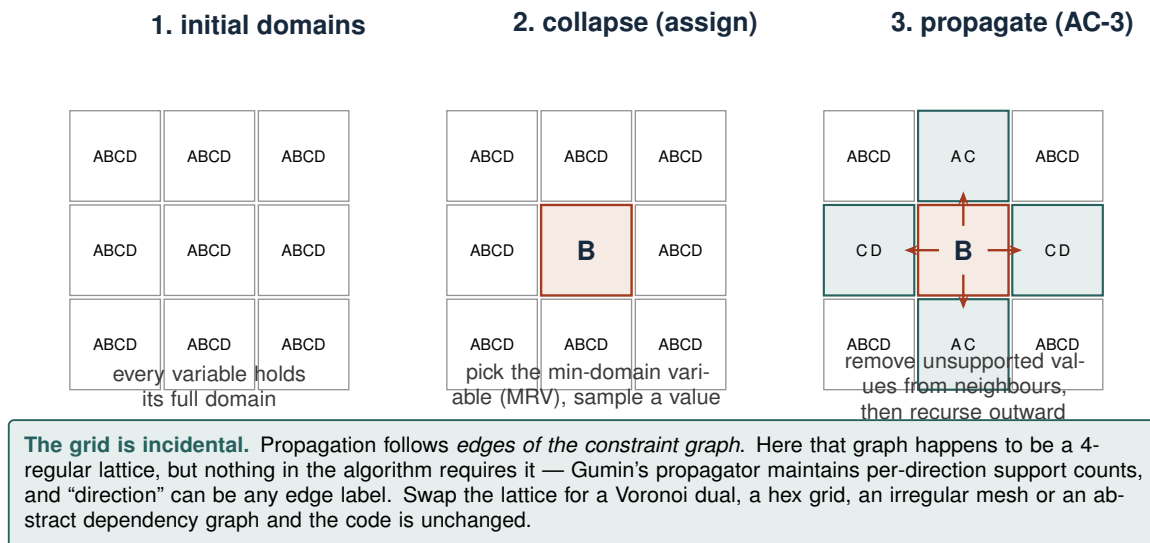


Figure 6.1. Collapse is assignment; propagation is arc consistency. Gumin's implementation maintains, for each cell/tile/direction triple, a count of how many tiles in the neighbouring cell still support it; when a count reaches zero the value is removed and the removal propagates. That is precisely AC-3's support-counting formulation (AC-4, strictly). The efficiency of the reference implementation comes from this bookkeeping, not from anything quantum.

The next level up, **path consistency**, reasons about triples and would eliminate most contradictions. It is not used because it is dramatically more expensive.

So contradictions are structural, not accidental. Gumin's reference implementation responds by discarding all work and restarting from scratch. That is a legitimate engineering trade for small outputs — it keeps the implementation tiny — but it is a weak response, and it is the first thing to change when scaling up.

At the bench — Better responses to a contradiction, in increasing order of implementation cost:

- **Chronological backtracking.** Undo the last assignment, try another value. Requires a trail of domain changes — perhaps forty lines. Turns most restarts into a few hundred microseconds of backtracking.
- **Backjumping / conflict-directed backjumping.** Jump to the assignment actually responsible for the wipeout rather than the most recent one.
- **Clause learning (CDCL).** Record *why* the contradiction happened so the same dead end is never re-entered. This is what turned SAT solving from a toy into industrial infrastructure, and it applies here unchanged.
- **Modification in blocks.** Merrell's model synthesis approach: generate in overlapping regions with the surroundings frozen, so any failure is bounded in space and cannot force a global restart. Often the best practical answer for large outputs.

6.3 Your Sudoku case, precisely

Sudoku is the cleanest possible instance, and worth stating exactly because the numbers are small enough to check.

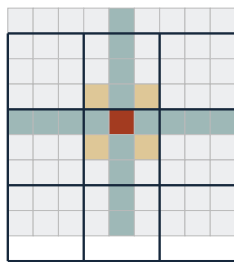
The constraint graph has 81 vertices. Each cell is constrained against the other 8 in its row, the other 8 in its column, and the other 8 in its 3×3 box — but the box overlaps the row and column in 4 cells that have already been counted. So each cell has

$$8 + 8 + (8 - 4) = 20$$

distinct neighbours, and the graph is **20-regular** with $81 \times 20/2 = 810$ edges (Figure 6.2).

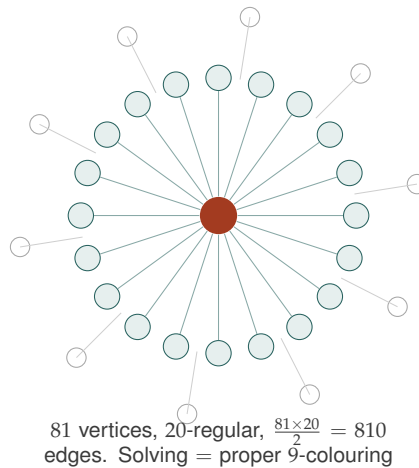
Solving a Sudoku is **proper 9-colouring of that specific 20-regular graph**, with some vertices pre-coloured. WFC is one propagation strategy over it — and not a particularly strong one, since human techniques such as naked pairs and hidden pairs are exactly the higher-order consistency that plain arc consistency omits.

the 20 neighbours of one cell



■ the cell

as a 20-regular constraint graph



Human Sudoku technique is a propagation-strength ladder, and WFC sits near the bottom of it.

Naked single = a domain of size 1. *Hidden single* = a value with one remaining support in a unit — this is already **beyond** plain arc consistency, since it reasons about a whole clique rather than a pair. *Naked/hidden pairs and triples* = higher-order consistency over k -subsets. *X-wing, swordfish* = reasoning over cycles in the constraint graph.

A WFC-style solver that only does AC-3 will hit contradictions on puzzles a competent human solves without guessing. If your Sudoku generator backtracks more than you expect, this — not the RNG — is why.

Figure 6.2. Sudoku is graph colouring, and WFC is one way to attempt it. The 20-regular structure is exact: row and column contribute 8 each, and the box contributes only 4 new neighbours because it shares 4 cells with the row and column already counted. Note the practical implication for a generator: because the graph is vertex-transitive under the Sudoku symmetry group, every cell has identical local structure, so min-entropy tie-breaking does almost no useful work early on and the seeded RNG dominates the first several collapses.

6.4 WFC on arbitrary graphs

Because propagation follows edges and nothing else, the grid is a special case rather than a requirement. Figure 6.3 shows the generalisation — and this is the version worth implementing, since it costs nothing extra and removes an artificial limitation.

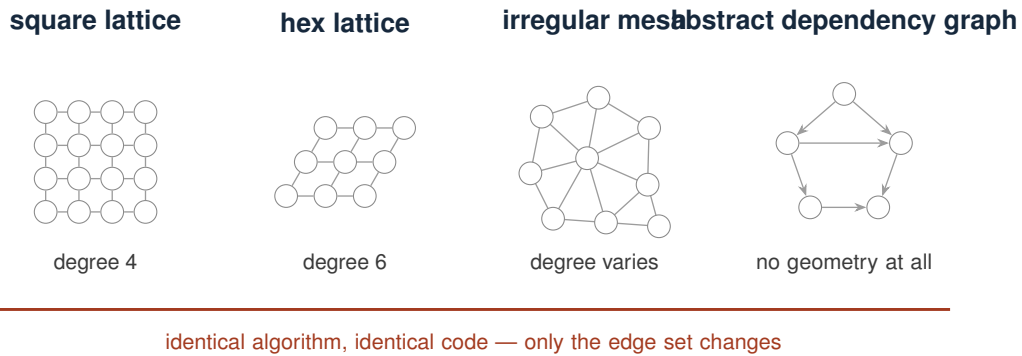


Figure 6.3. Grids are a special case, not a requirement. If the propagator is written against an edge list with labelled directions rather than against (x, y) offsets, all four of these work unchanged. The right-hand case is worth noting: with no geometry at all, WFC becomes a generic constrained-assignment engine — useful for quest structure, dialogue trees, or level connectivity rather than tile placement.

6.5 Where WFC sits in the family

WFC is not *sui generis*. It occupies a specific spot in a well-mapped landscape (Figure 6.4), and knowing which spot tells you what to reach for when it is not good enough.

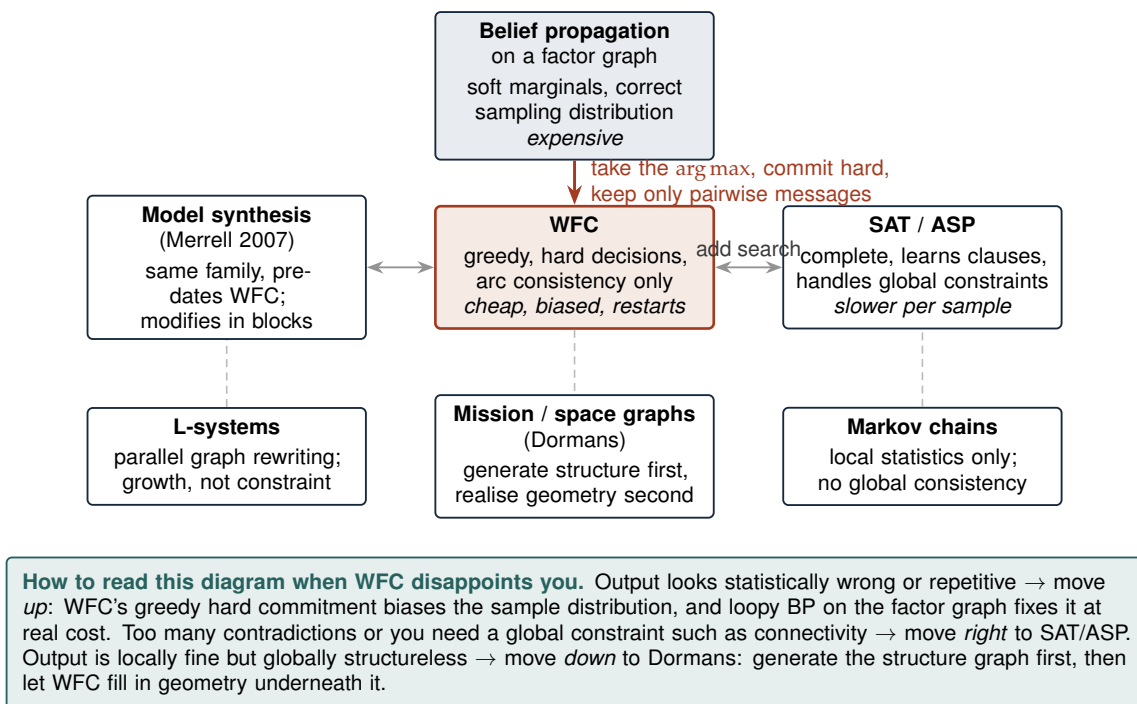


Figure 6.4. WFC as a greedy approximation to belief propagation. Karth & Smith's *WaveFunctionCollapse is Constraint Solving in the Wild* (2017) is the paper that made this framing explicit and is worth reading before writing a fourth custom propagator. The practical value of the map is diagnostic: each direction of movement corresponds to a specific class of dissatisfaction with the output.

Glossary: Wave Function Collapse and Procedural Generation

Terms introduced in this chapter

Wave Function Collapse (WFC) — Gumin’s generation algorithm: maintain a domain per cell, repeatedly collapse the minimum-entropy cell and propagate. Despite the name it contains no physics — it is arc consistency with a heuristic.

Superposition — WFC’s term for a cell’s remaining domain, i.e. the set of still-admissible tiles.

Collapse — Assigning one value to a cell, usually sampled from the remaining domain weighted by tile frequency.

Propagation — Removing now-unsupported values from neighbouring domains and recursing. Gumin’s implementation maintains per-direction support counts; this is AC-3.

Entropy heuristic — Choosing the cell with the fewest remaining options (weighted by tile probabilities). Identical to the classical *minimum remaining values* variable-ordering heuristic.

Contradiction — A cell whose domain becomes empty. Because WFC enforces only arc consistency and not path consistency, contradictions are reachable; the reference implementation restarts from scratch rather than backtracking.

Path consistency — The stronger condition on triples of variables. Enforcing it would eliminate most contradictions at substantially higher cost — the trade WFC declines to make.

Model synthesis — Merrell’s 2007 algorithm, predating WFC and in the same family, generating from an example model under adjacency constraints with an explicit modification-in-blocks scheme.

Factor graph — A bipartite graph of variables and factors encoding a joint distribution. WFC is recoverable as a greedy, hard-decision approximation to belief propagation on one.

Belief propagation (BP) — Message passing that computes (approximate) marginals on a factor graph. Running loopy BP instead of WFC gives better-distributed samples at materially higher cost.

Graph colouring — Assigning labels so adjacent vertices differ. Sudoku is proper 9-colouring of a fixed 81-vertex, 20-regular graph; WFC is one propagation strategy over it.

L-system — A parallel string- or graph-rewriting system with an interpretation step. The other major procedural-generation family, and equally a graph grammar.

Mission graph / space graph — Dormans’ two-stage dungeon generation: generate an abstract graph of gameplay dependencies, then a separate grammar realises it as geometry. Separating what must happen from where it happens.

CHAPTER 7

Dataflow, TouchDesigner and Hands

Two questions here, and they have different characters. *Is TouchDesigner graph-based?* — unambiguously yes, with one distinction that determines whether it is the right tool for you. *Can you program and interact with graphs using video and your hands?* — yes, and the hard part is not the tracking. It is a filter-design problem, which returns us to Chapter 3.

7.1 TouchDesigner is a typed dataflow DAG

TouchDesigner is a directed acyclic graph of typed operators, evaluated once per frame. Two properties are worth stating precisely because they drive everything else about working in it.

Evaluation is pull-based. A node does not compute when its input changes. It computes when something *downstream* asks for its output and it is marked dirty. Parameter changes propagate a dirty flag forward; the actual cooking happens backwards from whatever is being rendered. A branch nothing looks at costs nothing.

Wires are family-typed. TOP (texture, GPU), CHOP (channel/time-series), SOP (3D geometry), DAT (text and tables), MAT (material), COMP (component). You cannot wire a TOP into a CHOP; crossing families requires an explicit conversion operator, and those conversions are where the frame budget goes.

Cycles are illegal, so feedback needs an explicit one-frame delay (Figure 7.1).

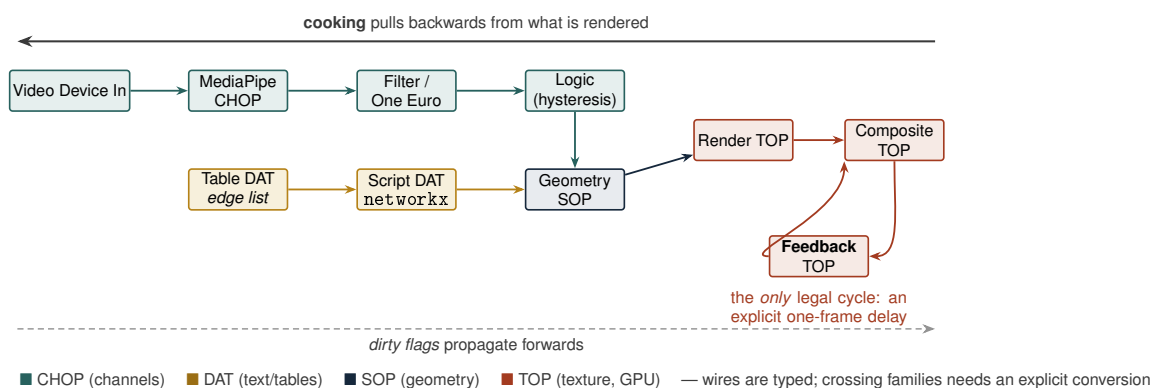


Figure 7.1. TouchDesigner's execution model. Dirty flags travel forwards; cooking pulls backwards from whatever is being displayed, so unviewed branches are free. The Feedback TOP is the sanctioned way to break acyclicity, and it is exactly a z^{-1} delay element — the same construct that makes a recurrent network trainable and a digital filter realisable.

7.1.1 The distinction that actually matters

In TouchDesigner the graph is the **program**, not the **data**. There is no native graph datatype. If you want to manipulate graph *structures* — build them, rewire them, run algorithms on them — you hold an edge list in a Table DAT and drive it from Python.

That is not a criticism, but it does decide the tool choice. TouchDesigner embeds CPython, so `networkx` works fine and the rendering is excellent. It is a strong *host and renderer* for graph work and a weak *graph engine*.

At the bench — If the graph-as-data aspect is central rather than incidental:

- **Houdini** — SOPs are genuinely topology-native (points, vertices, primitives are first-class and queryable), and PDG/TOPs exposes an execution dependency graph as manipulable data. The closest thing to a graph engine with a good renderer attached.
- **vvvv gamma** — strongly typed .NET dataflow; the closest of these to real programming, and comfortable if you want actual data structures.
- **Cables.gl** — browser-native, which matters if the target is the same environment as your existing `hollow.html` work. Lower ceiling, far lower friction.
- **Plain Python** plus a rendering layer, if the interaction is the point and the patching is not.

Given that your Sudoku WFC work already lives in the browser, the cheapest experiment is probably a browser-side hand-tracking layer over what you have, rather than porting into TouchDesigner to get hands.

7.2 Hands from video: the pipeline

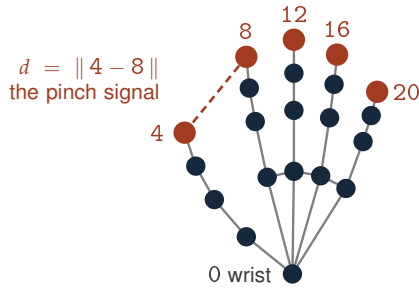
The tracking itself is close to solved and is not where the difficulty lives.

Option	Rough cost	Character
MediaPipe Hands/Tasks	free, any webcam	21 landmarks per hand, real-time on CPU, runs in-browser via WASM. The default starting point.
Ultraleap Controller 2	a few hundred dollars	Dedicated IR stereo. Substantially more robust than webcam tracking, native CHOP in TouchDesigner, and raw IR image access if you want to work below the SDK.
Depth camera (RealSense, Orbbec)	mid-range	True 3D and trivial background segmentation. Heavier to integrate; worth it if you need real depth rather than inferred.

For TouchDesigner specifically, Torin Blankensmith's *MediaPipe for TouchDesigner* delivers landmarks straight into CHOP channels and saves you building an OSC bridge.

7.3 The real problem is filter design

Landmarks jitter. They jitter even when the hand is perfectly still, because per-frame inference is independent and noise is not temporally correlated. The obvious fix — a low-pass filter — immediately presents the classic dilemma:



21 landmarks $\times$ 3 coordinates = 63 numbers per hand, per frame.

This looks like an enormous input bandwidth and is not. The landmarks are heavily correlated — finger joints are mechanically coupled, and the true dimensionality of comfortable hand posture is closer to *six to eight* than to sixty-three.

Two practical consequences: (i) run PCA on your own recorded poses rather than designing against raw landmarks; (ii) do not design gestures that require independent ring- and middle-finger motion, because most people physically cannot.

Figure 7.2. The landmark model. Twenty-one keypoints: wrist, then four per digit, tip landmarks marked. Thumb-tip to index-tip distance is the conventional pinch signal and is the running example for the rest of this chapter. Note that the nominal 63 degrees of freedom vastly overstate the usable input dimensionality.

- **Low cutoff:** smooth when still, unacceptably laggy when moving.
- **High cutoff:** responsive when moving, visibly jittery when still.

A fixed Kalman filter has the same problem in a more expensive package, since you are still committing to fixed process noise.

The **One Euro filter** (Casiez, Roussel & Vogel, CHI 2012) refuses the choice. It is a first-order low-pass whose cutoff frequency is a function of the estimated speed:

$$f_c = f_{c_{\min}} + \beta |\dot{\hat{x}}|$$

Still hand, low speed, low cutoff, heavy smoothing. Moving hand, high speed, high cutoff, minimal lag. Two parameters, about thirty lines of code, no matrices.

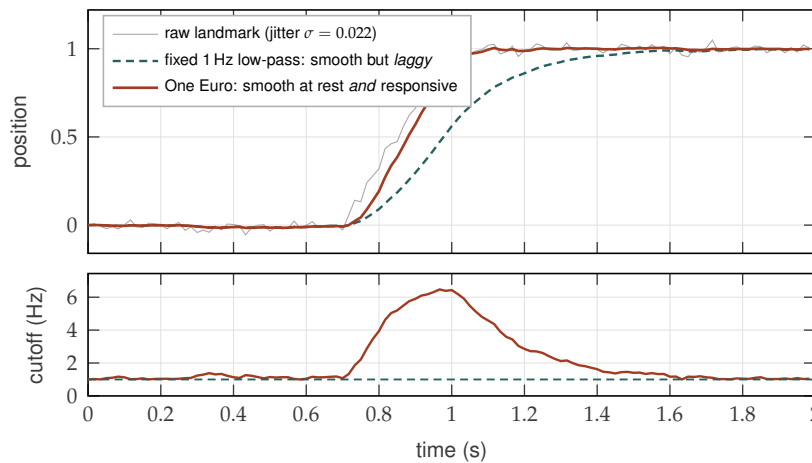


Figure 7.3. Measured, not asserted. A simulated landmark holds still, moves quickly over 300 ms, then holds still again, with Gaussian jitter of $\sigma = 0.022$ added. Upper panel: the fixed 1 Hz low-pass and the One Euro filter are *equally smooth at rest* — residual standard deviation 0.0049 against 0.0054, indistinguishable — but during the movement the fixed filter’s mean tracking error is 0.273 against One Euro’s 0.096, an improvement of about $2.8\times$. Lower panel: the adaptive cutoff (solid) rising with speed while the fixed cutoff (dashed) does not. **The trade is not smoothness against responsiveness — it is strictly better on the axis that matters, for two parameters and thirty lines.**

At the bench — Tuning, in order: set $\beta = 0$ first and lower $f_{c_{\min}}$ until the hand is acceptably still at rest — this is the jitter/lag trade in isolation. Then raise β until fast movement no longer lags. Do not tune both at once; they are separable and tuning them jointly is how people conclude the filter does not work.

7.4 Discrete decisions need hysteresis

Continuous filtering is only half of it. The moment you threshold a continuous signal into a discrete event — pinch, grab, select — you get *chatter*: the signal hovers at the threshold and the state flips repeatedly, firing dozens of events per second.

The fix is standard control engineering: separate enter and exit thresholds, plus a dwell time (Figure 7.4).

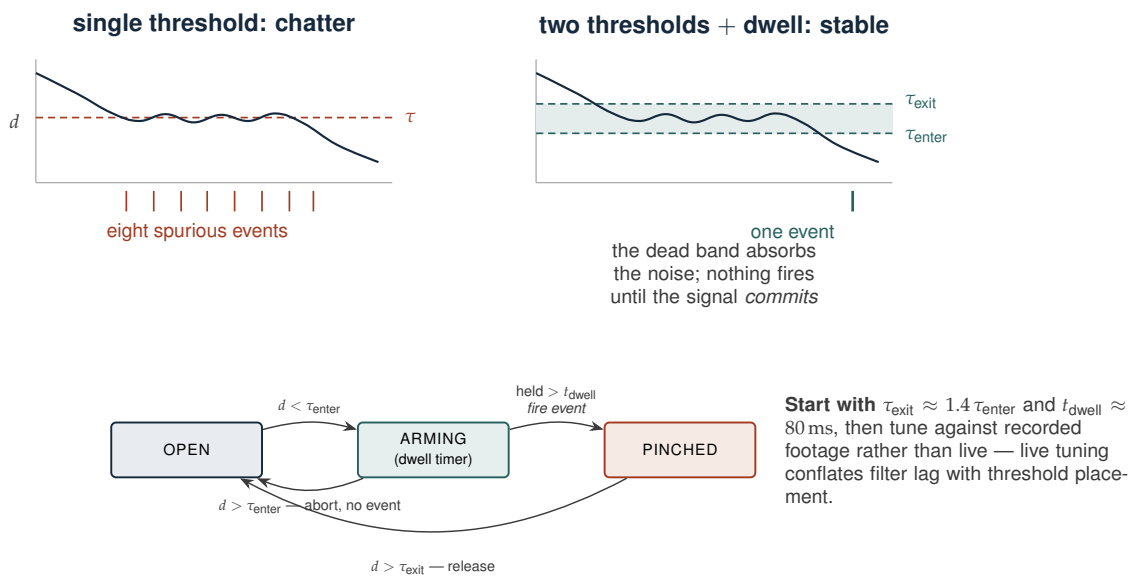


Figure 7.4. Every discrete gesture derived from a continuous signal needs hysteresis. A single threshold turns unavoidable noise near the boundary into a burst of spurious events. Two thresholds create a dead band that noise cannot cross, and a dwell timer requires the signal to commit before an event fires. The state machine is another small labelled digraph — the same object as the Arabic joining automaton in Figure 5.1, doing an entirely different job.

7.5 The honest design constraint

Two facts about mid-air interaction that no amount of engineering removes, and that are better designed around than fought.

There is no haptic feedback. Your hand cannot feel a boundary it has crossed, so you do not know which mode the system is in without looking. Mode errors are therefore constant rather than occasional. This is why hysteresis and dwell are not polish — they are what makes the interaction usable at all — and why continuous visual feedback of the current state is mandatory, not optional.

Gorilla arm is biomechanical. Sustained unsupported mid-air interaction becomes fatiguing within a few minutes. This is a property of shoulder abduction under static load,

not of user acclimatisation, and it does not improve with practice. Design for hands low and supported, in short bursts.

Hands are excellent for **continuous spatial manipulation** of a graph — layout, camera, parameter scrubbing, pulling nodes around in 3D — and poor for **precise topological authoring** — connect this specific output port to that specific input port.

The route around this is not to abandon the idea. It is **hybrid input**: send the continuous/spatial channel to the hands and the symbolic/precise channel to keyboard, voice or a physical control surface (Figure 7.5). That split is what ships in the systems that work.

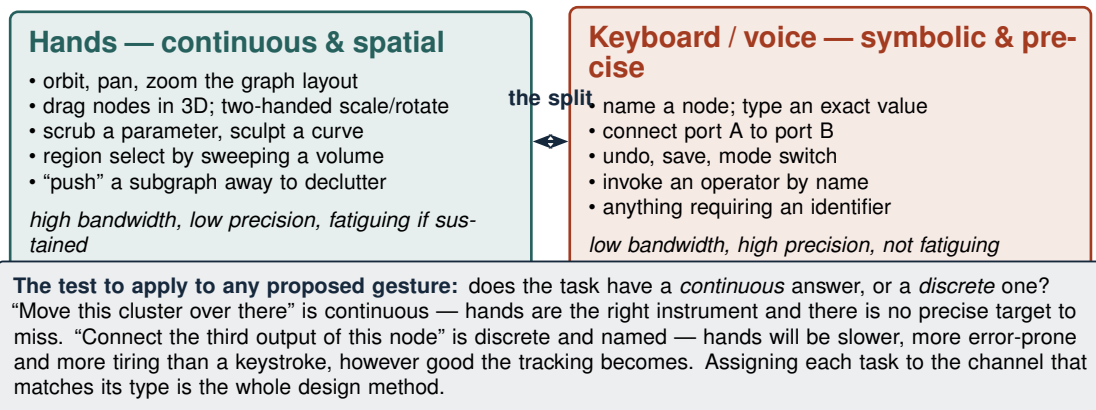


Figure 7.5. Match the channel to the type of the task. The failure mode of gestural interfaces is not bad tracking; it is asking the continuous channel to do symbolic work. Hands and keyboard are complementary rather than competing, and a system that uses both will beat one that insists on either alone.

Glossary: Dataflow, TouchDesigner and Hands

Terms introduced in this chapter

Dataflow graph — A program represented as nodes (operations) and edges (data dependencies). TouchDesigner, Houdini, vvvv, Max and Pure Data are all instances.

Cooking — TouchDesigner’s term for evaluating a node. Cooking is *pull-based*: a node cooks only when something downstream requests its output and it is marked dirty.

Dirty propagation — Marking a node and its dependents stale when a parameter or input changes, so the next frame recomputes only the affected subgraph.

Operator families — TOP (2D texture, GPU), CHOP (channel/time-series), SOP (3D geometry), DAT (text and tables), MAT (material), COMP (component). Wire types are family-typed; conversion needs an explicit bridge operator.

Feedback TOP — The explicit one-frame delay node that makes recurrence legal in an otherwise acyclic graph. Any cyclic dataflow must be broken by an explicit delay.

Graph as program vs graph as data — TouchDesigner’s graph is its *program*; it has no native graph datatype. Graph *structures* live in a DAT edge list driven from embedded Python. Houdini’s PDG/TOPs, by contrast, exposes an execution dependency graph as manipulable data.

Hand landmark — One of MediaPipe’s 21 keypoints per hand (wrist, plus 4 per digit), returned with normalised image coordinates and a relative depth estimate.

Jitter — High-frequency noise in tracked landmark position, present even when the hand is still. A signal-processing problem, not a tracking-quality problem.

One Euro filter — Casiez et al. (CHI 2012): a first-order low-pass filter whose cutoff frequency rises with estimated speed, giving smoothing at rest and responsiveness in motion. Two parameters (f_{cmin} , β); roughly thirty lines of code.

Cutoff frequency — The -3 dB point of a low-pass filter. Fixed low cutoff means smooth but laggy; fixed high cutoff means responsive but jittery. The One Euro filter refuses the choice by making it a function of speed.

Hysteresis — Using different thresholds for entering and leaving a state, so a signal hovering at the boundary cannot chatter. Mandatory for any discrete gesture detected from a continuous quantity.

Dwell time — A required minimum duration in a state before a transition commits. Combined with hysteresis, this is what makes mid-air pinch detection usable.

Gorilla arm — Fatigue from sustained unsupported mid-air interaction, measurable within minutes. A biomechanical constraint, not a matter of user acclimatisation: design for hands low and at rest, in short bursts.

Mode error — Acting in the belief that the system is in a different state. Endemic to mid-air input because there is no haptic feedback to signal which mode the hand is in.

Hybrid input — Splitting interaction so the continuous/spatial channel goes to hands and the symbolic/precise channel goes to keyboard or voice. The split that actually ships.

CHAPTER 8

One Bit: Dithering, Halftones and Automata

Reducing an image to one bit per pixel destroys almost all of it. What survives is determined entirely by one decision, and that decision turns out to be the same one Chapter 3 was about.

Quantisation destroys information. Dithering does not recover any of it. Dithering **moves the resulting error in frequency space** — out of the band the eye resolves and into the band it does not.

Dithering is *spectral shaping of quantisation error*. It is a filter design problem, and everything good about it follows from that.

This makes the human visual system part of the specification. The contrast sensitivity function is band-pass, peaking around 3 cycles/° to 5 cycles/° and falling away sharply above about 30 cycles/°. So the target is explicit: **put the error above the cutoff**. Noise with power rising towards high frequency is called *blue noise*, and it is what every good dither is trying to produce.

8.1 Four quantisers, one test image

Figure 8.1 runs the same gradient through four methods. Figure 8.2 explains why they look so different, and the explanation is not aesthetic — it is measurable.

Sanity check — The claim that total error is conserved and only its *distribution* changes is worth verifying rather than believing. Compute `np.abs(out - img).mean()` for each method: the values are close, while the spectra differ by two orders of magnitude in the visible band. If a dithering method appears to reduce total error, it is doing tone mapping as well and the comparison is not clean.

8.2 Error diffusion is a filter on a directed graph

Floyd–Steinberg reads as a piece of folklore — some magic fractions with denominator 16 — until it is drawn properly. Then it is obviously a stencil, and Figure 8.3 is deliberately the same drawing as Figure 2.3.

8.2.1 Ordered dithering: stateless, parallel, spiky

The Bayer matrix is built by recursive doubling (Figure 8.4), which makes it self-similar — and self-similarity in space means concentrated harmonics in frequency. Hence the visible

plain threshold

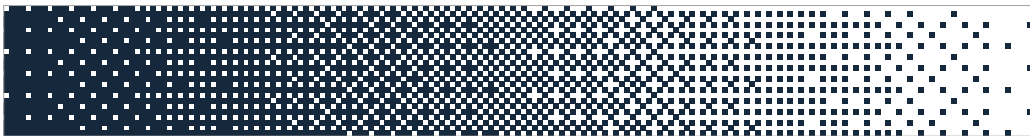


all error at low frequency: hard banding

white-noise dither



error flat across all frequencies: uniform grain

ordered (Bayer 8×8)

error at fixed frequencies: visible cross-hatch

Floyd–Steinberg



error pushed high: the eye low-passes it away

Figure 8.1. The same linear ramp, quantised four ways, drawn as vector fills. Plain thresholding produces a single hard edge — all the error has gone into one enormous low-frequency feature. White-noise dithering removes the banding but puts error everywhere, including where the eye is most sensitive. Ordered dithering is clean but leaves the tile’s own harmonics visible as cross-hatch. Floyd–Steinberg produces the finest texture. These are the same four algorithms measured in Figure 8.2.

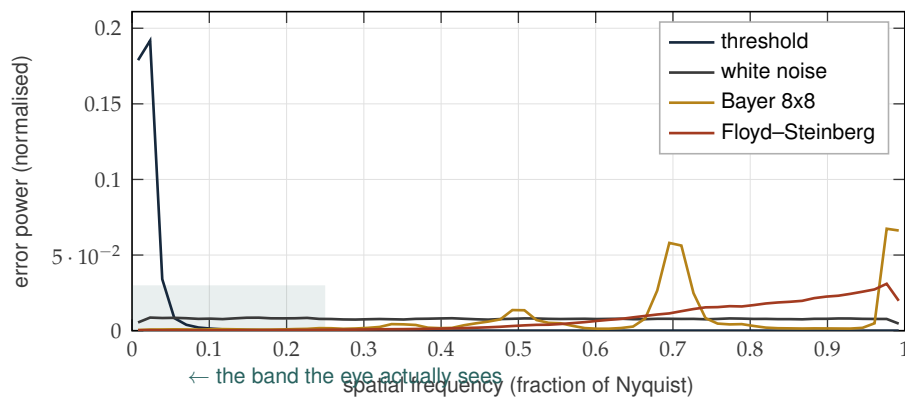


Figure 8.2. The whole argument, measured. Radially averaged power spectrum of the quantisation error (output – input) on a smooth 256×256 test field, DC removed. The shaded region is roughly the band the eye resolves at normal viewing distance. Fraction of total error energy falling in that visible band: **threshold** 86.3 %, white noise 25.3 %, Bayer 8×8 2.7 %, **Floyd–Steinberg** 0.9 %. That is a factor of roughly 96 between the worst and the best — and note that *the total error energy is essentially the same in all four cases*. Nothing was removed. It was moved.

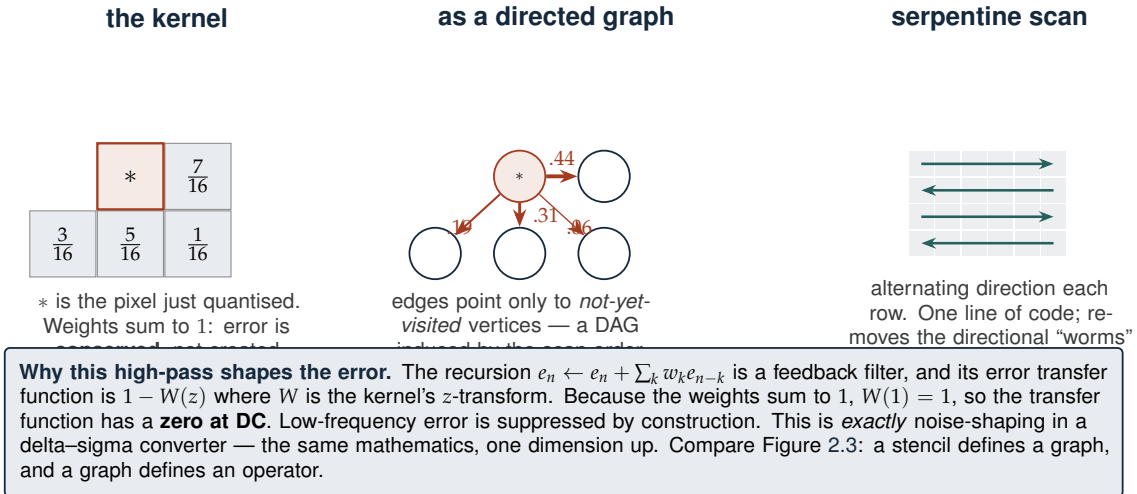


Figure 8.3. Error diffusion is a recursive filter whose support is a graph neighbourhood.

The three drawings are the same object, exactly as in Chapter 2. The important structural point is the third: the kernel's weights summing to unity forces a zero at DC in the error transfer function, which is *why* the error comes out blue. It is not a lucky property of these particular fractions.

cross-hatch, and hence Bayer's 2.7% in the visible band being worse than Floyd–Steinberg's 0.9% despite both looking “fine”.

But it has two properties error diffusion cannot match, and they matter enormously in Chapter 9:

- **Stateless and parallel.** Every pixel is independent, so it is one texture lookup in a fragment shader. Error diffusion is inherently sequential and hostile to GPUs.
- **Temporally stable.** A fixed threshold field gives the same answer for the same input every frame.

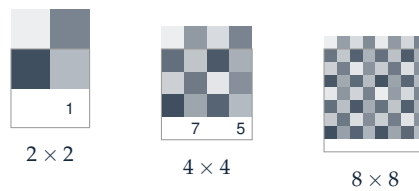


Figure 8.4. $M_{2n} = \begin{pmatrix} 4M_n & 4M_n+2 \\ 4M_n+3 & 4M_n+1 \end{pmatrix}$. Each doubling interleaves four scaled copies of the previous matrix, so the structure is exactly self-similar across scales. That is the source of both its virtue — perfectly even distribution of thresholds — and its defect, since a self-similar spatial structure has a spiky rather than smooth spectrum. **Void-and-cluster** (Ulichney, 1993) fixes this by abandoning the closed form: it *optimises* a mask by repeatedly locating the largest void and the tightest cluster and swapping them. That is constraint satisfaction over a spatial objective — Chapter 6 machinery applied to a threshold field.

At the bench — Halftoning is the same problem with a different output device. Classical *AM screening* holds dot spacing fixed and varies dot size; *FM screening* holds size fixed and varies placement, which is dithering under another name. The famous process-colour screen angles (15°, 75°, 45°, 0° for C, M, K, Y) exist because

superimposing periodic patterns produces beat frequencies — moiré — and those particular angles push the beats out of the visible band. That is Chapter 3’s aliasing argument, solved in a print shop a century before it had that name.

8.3 Cellular automata are message passing

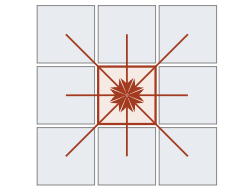
A cellular automaton is a graph with a state at each vertex, updated synchronously by a rule that reads only the local neighbourhood. Written that way the identification is immediate.

A cellular automaton is a **graph neural network with a lookup table instead of learned weights**, run for many steps with shared parameters.

Totalistic rules — those depending only on the sum of neighbour states, such as Conway’s Life — are the case where the aggregation function is permutation-invariant. That is precisely the constraint that defines a message-passing layer in Chapter 2.

Figure 8.5 lays the correspondence out.

Moore neighbourhood



an 8-regular graph. Nothing requires it to be a grid

Cellular automaton

$$s_i^{t+1} = R\left(s_i^t, \sum_{j \in N(i)} s_j^t\right)$$

R is a lookup table. States discrete, time discrete, parameters shared across all steps.

Message-passing layer

$$h_i^{t+1} = \phi\left(h_i^t, \bigoplus_{j \in N(i)} \psi(h_j^t)\right)$$

ϕ is a learned MLP. States continuous, but otherwise the same equation.

The ladder, and where it leads

Discrete states, table rule → classical CA. Life, Rule 110, both Turing-complete.

Continuous states, table rule → Lenia, SmoothLife. Smooth gliders, organic motion.

Continuous states, learned rule → Neural CA (Mordvintsev et al., 2020): a GNN trained to grow a target image from a single seed.

Continuous states and time → reaction–diffusion, i.e. a PDE. Which is Ch. 2, and the next section.

Figure 8.5. One equation, two literatures. The CA and GNN update rules differ in whether the aggregation is a lookup or a learned function, and in nothing else structurally. The practical consequence runs in both directions: CA rules can be run on arbitrary graphs (Voronoi cells, meshes, molecular graphs), and GNN theory — oversmoothing, receptive field growth, the K -hop locality of Chapter 3 — applies directly to CA. A CA’s “light cone” after t steps is precisely the t -hop ball of Figure 3.3.

8.4 Turing patterns select a Laplacian eigenvector

This is the tightest connection in the chapter, and it closes the loop back to Chapter 2 completely.

Take a two-species reaction–diffusion system,

$$\partial_t u = D_u \nabla^2 u + f(u, v), \quad \partial_t v = D_v \nabla^2 v + g(u, v)$$

linearise about a uniform steady state, and look for solutions $\propto e^{\sigma t} e^{ikx}$. Since $\nabla^2 e^{ikx} = -k^2 e^{ikx}$ — the same eigenrelation that started Chapter 3 — the growth rate is the largest eigenvalue of

$$\mathbf{M}(k) = \mathbf{J} - k^2 \mathbf{D}, \quad \mathbf{J} = \begin{pmatrix} f_u & f_v \\ g_u & g_v \end{pmatrix}, \quad \mathbf{D} = \text{diag}(D_u, D_v)$$

If the uniform state is stable without diffusion but $\sigma(k) > 0$ for a band of k , the system spontaneously forms stationary spatial pattern, and the dominant wavelength is $2\pi/k_{\max}$ (Figure 8.6).

On a graph, $-\nabla^2$ becomes $\mathbf{L}$, so k^2 becomes $\lambda_{\text{Laplacian}}$. The dispersion relation is then a function of the graph's spectrum, and

the pattern that forms is the Laplacian eigenvector whose eigenvalue has the largest growth rate.

Pattern formation is **eigenvector selection**. Change the graph and you change the available modes, so you change what can grow — animal-coat markings, Chladni figures on a plate, and the resonant modes of a drum are all the same statement about a spectrum.

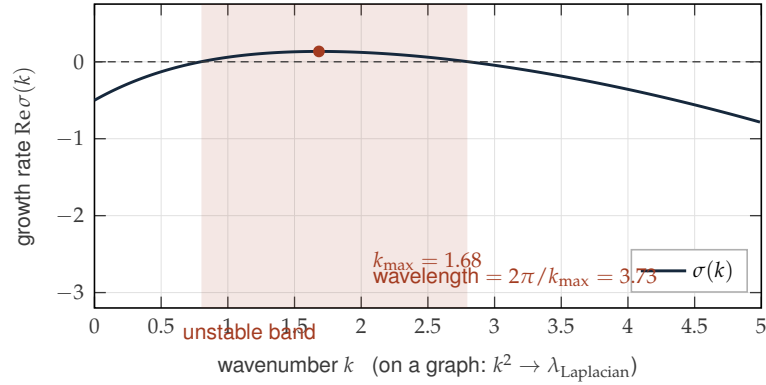


Figure 8.6. Diffusion-driven instability, computed for a real parameter set ($f_u = 0.5$, $f_v = -1$, $g_u = 1$, $g_v = -1.5$, $D_u = 0.05$, $D_v = 1$; all four Turing conditions verified numerically). The uniform state is stable at $k = 0$ and stable again at large k , but unstable across the shaded band $k \in [0.80, 2.80]$, with the fastest-growing mode at $k_{\max} = 1.68$ giving a pattern wavelength of 3.73. **Note what the horizontal axis becomes on a graph:** $k^2 \rightarrow \lambda_{\text{Laplacian}}$, so this curve is a filter applied to the graph spectrum — a band-pass filter, in exactly the sense of Chapter 3. Turing patterns are a band-pass filter on the Laplacian eigenbasis, with the gain supplied by chemistry instead of a convolution kernel.

The gotcha — Reaction–diffusion is stiff, and this is the failure mode that wastes the afternoon. The inhibitor’s diffusion constant must substantially exceed the activator’s — typically by an order of magnitude or more — and that ratio is exactly what sets the stability limit of an explicit integrator. For forward Euler on a grid of spacing h the constraint is

$$\Delta t < \frac{h^2}{4 D_{\max}}$$

so the *faster* species dictates the timestep for the whole system. Exceed it and the field does not degrade gracefully — it explodes to NaN within a few dozen steps, usually after appearing to work.

The mechanism is Chapter 2’s: an explicit scheme is unstable when $\Delta t \lambda_{\max}(\mathbf{L}) > 2$, and $\lambda_{\max}$ scales as $D_{\max}/h^2$. The levers, in order of effort: shrink Δt (cheap, slow); use an implicit or semi-implicit scheme for the diffusion term, which is unconditionally

stable and is what any serious solver does; or split the operators and solve diffusion in the Fourier domain where it is exactly diagonal. *If your reaction–diffusion blows up, do not tune the chemistry — check the timestep against that inequality first.*

8.5 Why one bit produces such a strong aesthetic

The constraint is severe, and severity is the point. With one bit there is no tonal channel at all, so *every* tonal decision is forced into the spatial domain. Texture stops being decoration and becomes the only available carrier of value.

That is why the look is so recognisable and so hard to fake with a post-process filter: Macintosh QuickDraw and HYPERCARD, teletext and PETSCII, the Playdate’s 1-bit memory LCD, *Return of the Obra Dinn*. The aesthetic is not nostalgia for old hardware. It is what happens when the tonal axis is removed and the spatial axis has to carry everything.

Chapter 9 takes this apparatus and points it at moving images and at data.

Glossary: One Bit: Dithering, Halftones and Automata

Terms introduced in this chapter

Quantisation — Mapping a continuous value to one of finitely many levels. Information is destroyed; the only remaining decision is *where the resulting error goes* in frequency space.

Dither — Noise added *before* quantisation. Counter-intuitively it improves the result, because it decorrelates the error from the signal, converting structured banding into noise that can then be spectrally shaped.

Blue noise — Noise whose power spectrum rises with frequency, i.e. energy concentrated above the band the eye resolves. The target distribution for any perceptually good dither.

Error diffusion — Quantise a pixel, then push the resulting error into not-yet-visited neighbours according to a kernel. A recursive filter on a directed graph; the recursion is what high-pass shapes the error.

Floyd–Steinberg — The canonical error-diffusion kernel: 7/16 ahead, then 3/16, 5/16, 1/16 on the row below. Weights sum to 1, so error is conserved rather than created.

Serpentine scan — Alternating scan direction each row. Costs one line of code and removes the directional worm artefacts that a uniform left-to-right raster produces.

Ordered dither — Thresholding against a fixed tiled matrix. Stateless, parallel, and temporally stable — but its error sits at the tile’s harmonics, so the texture is visible as structure rather than grain.

Bayer matrix — The recursively constructed ordered-dither threshold matrix, $M_{2n} = \begin{pmatrix} 4M_n & 4M_n+2 \\ 4M_n+3 & 4M_n+1 \end{pmatrix}$. Self-similar, which is exactly why its spectrum is spiky.

Void-and-cluster — Ulichney’s method for generating blue-noise masks: iteratively find the largest void and tightest cluster and swap. An explicit optimisation over a spatial objective — constraint satisfaction, not a formula.

Halftone — Reproducing continuous tone with a pattern of marks. *AM screening* varies dot size on a fixed grid; *FM screening* varies dot placement at fixed size, which is dithering by another name.

Moiré — Beat frequencies from superimposed periodic patterns — aliasing, in the sense of Ch. 3. The reason process colour uses screen angles of 15°, 45°, 75° and 0°.

Contrast sensitivity function (CSF) — The eye’s spatial-frequency response: band-pass, peaking near 3–5 cycles per degree and falling off sharply above roughly 30. The transfer function that dithering is engineered against.

Cellular automaton (CA) — A graph with a state per vertex updated synchronously by a local rule over its neighbourhood. Formally a discrete-time, discrete-state message-passing network — the same object as Ch. 2’s GNN, with a lookup table instead of learned weights.

Totalistic rule — A CA rule depending only on the *sum* of neighbour states, not their arrangement. Permutation-invariant aggregation — precisely the constraint that defines a graph neural network layer.

Reaction–diffusion — $\partial_t u = D \nabla^2 u + f(u, v)$: diffusion plus local nonlinear reaction. On a graph, $\nabla^2 \rightarrow -L$, so it is Ch. 2’s heat equation with a source term.

Turing pattern — Stationary spatial structure arising when diffusion *destabilises* a uniform state. Requires the inhibitor to diffuse faster than the activator.

Dispersion relation — Growth rate $\sigma(k)$ as a function of wavenumber. On a graph the substitution is $k^2 \rightarrow \lambda_{\text{Laplacian}}$, so the selected pattern is literally the Laplacian eigenvector with the largest growth rate.

One-bit graphics — Rendering to a two-level display. A constraint that forces every tonal decision into the spatial domain — which is why it produces such a strong and recognisable aesthetic.

CHAPTER 9

Rendering: Putting the Error Where You Want It

Chapter 8 established that dithering does not remove error, it relocates it. This chapter takes that idea seriously as a design principle and points it at moving images and at data.

Every rendering decision is a decision about **where to put unavoidable error**. Spatial frequency, temporal frequency, or structure — those are the only three places it can go, and a style is a routing policy over them. Which means an aesthetic is not a filter applied at the end. It is a choice of where in the spectrum to hide the damage — and choosing to *expose* the damage instead is equally available, which is most of what glitch art is.

9.1 The temporal axis, where naive implementations die

Take the best still-image ditherer you have and run it per frame on video. It will look terrible, and the reason is precise.

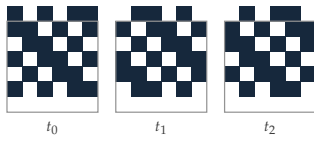
Floyd–Steinberg is optimised over (x, y) only. Applied independently to successive frames, a pixel whose input value barely changes can flip repeatedly, because error diffusion is a chaotic recursion — a tiny input perturbation propagates along the whole scan. The output is spatially excellent and *temporally white*. Broadband noise in t is exactly what the eye is most sensitive to. The image boils.

This is structurally the same problem as jittering hand landmarks in Chapter 7, and it has the same shape of solution. There, the fix was not to filter harder but to shape the filter’s response as a function of speed. Here, the fix is not to dither better per frame but to shape the error spectrum in (x, y, t) **jointly**.

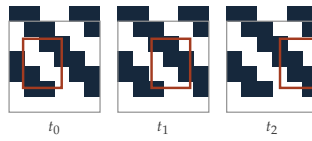
Figure 9.1 lays out the three strategies and what each one costs.

At the bench — Four practical options, in increasing order of cost and quality:

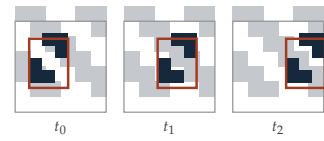
1. **Fixed ordered dither.** One texture lookup, perfectly stable, swims. The floor. Always start here to confirm the rest of the pipeline works.
2. **Golden-ratio frame offset.** Offset the threshold field by $n \bmod 1$ multiples of $\phi^{-1} \approx 0.618$ each frame. Nearly free, and it converts objectionable boiling into a fine shimmer that the eye integrates. Disproportionately effective for the effort.
3. **Spatiotemporal blue noise** (Wolfe et al., 2022). A precomputed 3D mask that is blue in (x, y, t) jointly rather than per-slice. Same runtime cost as (1) — it is still

A independent per frame

spatially optimal, temporally white. The pattern is uncorrelated frame to frame: it *boils*. Worst possible outcome despite the best per-frame algorithm.

B fixed screen-space field

stable, but the texture is nailed to the screen. The object (outlined) slides through a stationary pattern — *dither swim*.

C anchored to the surface

the pattern travels with the geometry. Index the threshold field by surface coordinates, not screen position. *Obra Dinn's* solution.

Figure 9.1. The temporal axis defeats naive per-frame dithering. Strategy A is what you get by calling a still-image ditherer in a loop, and it is the worst of the three despite using the best algorithm. B is stable but produces *swim* — the classic tell of one-bit rendering done quickly. C costs a UV lookup and fixes it. The general principle: **decide what the dither pattern is attached to**, because if you do not decide, it is attached to the screen.

one lookup — with a genuinely correct spectrum. If you only take one thing from this chapter, take this.

4. **Advected dither.** Transport the threshold field along the optical flow so it follows the content. Best-looking, most expensive, and needs a flow field — though an interframe codec is already carrying usable motion vectors for free.

The gotcha — Dither in linear light, not in gamma-encoded values. This is the single most common reason one-bit output comes out visibly too dark, and it is invisible until you compare against the source side by side.

The mechanism: sRGB stores luminance non-linearly, roughly $V = L^{1/2.2}$. A mid-grey of $L = 0.5$ is stored as $V \approx 0.73$. Dithering decides what fraction of pixels to set white, and that fraction is a *linear* average of light on the display — so thresholding against the encoded value 0.73 lights up far more pixels than the 50% the scene actually called for, or, depending on the direction of your error, far fewer. The output is systematically wrong in midtones while looking perfectly correct in pure black and pure white, which is exactly why it survives casual checking.

The fix is one line in each direction: linearise before quantising, and re-encode only for display. The check is one measurement: dither a 50% linear grey field and count the white pixels. It must come out at 50%. If it reads around 73%, you are dithering in the wrong space.

9.2 The design space

Once error routing is the frame, the techniques stop being a grab bag and arrange themselves along axes (Figure 9.2).

9.3 Three unusual routes, spelled out

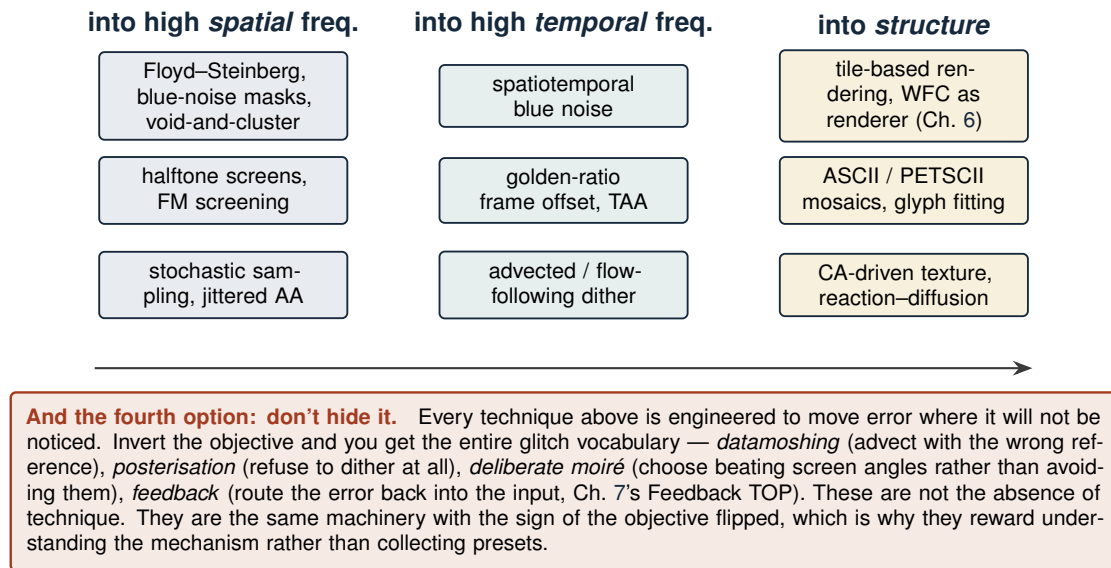


Figure 9.2. Techniques arranged by where they send the error. The value of the map is that it makes the unexplored cells visible. Reaction–diffusion driven by video luminance, or a WFC tileset whose adjacency rule is tonal continuity, are not standard techniques — they are empty cells in a grid that is otherwise well populated, and the grid is what suggests them.

9.3.1 WFC as a renderer

Chapter 6 treated WFC as a generator. It is equally a *constrained* renderer, and this is a genuinely under-explored corner.

Let the tile set be a library of small patches — dither cells, glyphs, brush marks, stamps of any kind. Give each tile an average density. Then impose two constraint families:

- **Tonal:** a tile's density must match the source image's local luminance to within a tolerance. This is a unary constraint, i.e. a per-cell domain restriction — it is how the image enters at all.
- **Adjacency:** neighbouring tiles must join without a seam. This is the binary constraint, and it is what makes the output look *drawn* rather than sampled.

The result is a renderer that produces coherent mark-making rather than independent per-pixel decisions, because the propagation in Figure 6.1 is doing the work that a human eye does when it keeps hatching consistent across a region.

At the bench — Two things to get right, both flagged earlier in the document. *First*, the tonal constraint must have a tolerance band, not an exact match — with an exact match the domains are usually singletons, propagation has nothing to do, and you have written an expensive nearest-neighbour lookup. *Second*, this is the temporal-coherence problem again: run WFC independently per frame and the mark-making reshuffles completely every frame. Seed each frame from the previous solution and re-solve only where luminance changed materially — which is Merrell's modification-in-blocks (Chapter 6) applied along the time axis.

9.3.2 Reaction–diffusion driven by the image

Chapter 8 showed that a Turing system selects a wavelength by picking the Laplacian eigenvalue with the largest growth rate. Now make the reaction parameters a function of local image luminance.

Bright regions get parameters yielding short wavelength; dark regions get long. The system then self-organises into a texture whose *spatial frequency* encodes tone — which is precisely what a halftone screen does, except that the pattern is emergent and organic rather than a regular grid. It is FM screening with the frequency chosen by chemistry.

This is worth the effort for one specific reason: the output has no preferred axis and no visible tile, so it does not read as “computer” the way ordered dithering does. It also sits naturally alongside biological subject matter, which given your other work may be the point.

9.3.3 Datamoshing, mechanically

Datamoshing is usually described as corruption. It is more precisely *error diffusion with a deliberately mismatched reference*.

An interframe codec stores I-frames (complete images) and P-frames (motion vectors plus residuals). A P-frame says: take these blocks from the previous frame, move them here, add this correction. Delete the I-frames and the motion vectors survive but the texture they transport is now whatever was on screen before.

The transport operator is intact; the thing being transported is wrong. That is advection — the same operation as option (4) in the temporal-coherence list above — applied to the wrong field. Which explains the characteristic look precisely: content smears *along real motion paths* rather than dissolving randomly, because the motion field is still correct. It is the only glitch technique that produces plausible fluid dynamics, and that is why.

9.4 It does not have to be an image

The pipeline takes a scalar field on a graph and returns a one-bit field on the same graph. Nothing anywhere requires that graph to be a pixel lattice — which Figure 6.3 already established for WFC and which applies here unchanged.

Field	Graph	What dithering it gives you
Plate luminance over time	pixel lattice $\times$ time	Growth as one-bit animation; texture density encodes density
Read depth along a genome	path graph	Coverage as a 1D dither; visually honest about sampling noise
Expression matrix	bipartite gene–sample graph	Heatmap without a false continuous colour ramp
Sensor array / bioreactor	irregular sensor graph	Spatial state at a glance, no interpolation invented
Any simulation on a mesh	the mesh itself	Halftone that respects the actual topology

There is a real epistemic argument here, not only an aesthetic one. A continuous colour ramp implies a precision the underlying measurement usually does not have, and it invents smooth interpolation between samples that were never taken. A dithered one-bit rendering is *visibly discrete*: the viewer can see the individual marks, and therefore see the sampling. It refuses to imply information that is not there. For open-science work streamed publicly, that is a defensible reason to choose it over a colourmap, quite apart from how it looks.

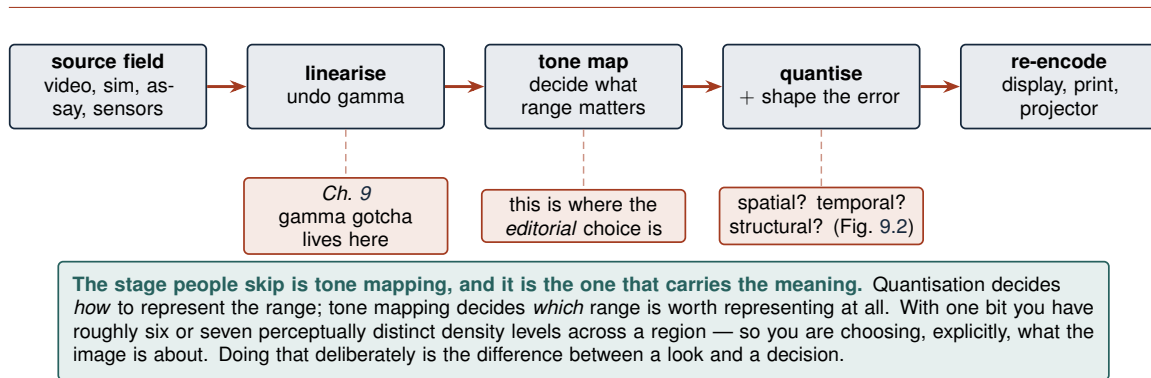


Figure 9.3. Where each decision sits. Errors introduced early cannot be repaired late: dithering in gamma space cannot be fixed by a better kernel, and a bad tone map cannot be rescued by a better dither. The pipeline is the same whether the input is video, a simulation on a mesh, or an assay — only the graph underneath changes.

9.5 The seams

It is worth stating plainly what this pair of chapters has actually done, because the method generalises past the subject matter.

Dithering belongs to printing. Halftones belong to photomechanical reproduction. Cellular automata belong to theoretical computer science. Reaction–diffusion belongs to mathematical biology. Error diffusion belongs to signal processing. One-bit graphics belongs to games and demoscene practice. These fields have separate journals, separate vocabularies, and very little citation traffic between them.

They are the same subject. Specifically:

Apparently separate	Actually
Error diffusion	a recursive filter with a zero at DC — delta–sigma modulation in 2D (Fig. 8.3)
Blue-noise mask generation	constrained optimisation over a spatial objective (Ch. 6)
Halftone screen angles	anti-aliasing; choosing where the beat frequencies land (Ch. 3)
Cellular automata	message passing with a lookup table (Ch. 2)
Turing pattern wavelength	the Laplacian eigenvector with the largest growth rate (Fig. 8.6)
Temporal dither coherence	the same jitter/lag trade as hand tracking (Fig. 7.3)
Datamoshing	advection with a mismatched reference field

Each of those right-hand entries is a place where technique developed in one field transfers immediately to another — and mostly has not, because the vocabularies do not match and nobody was looking. The Laplacian was the thread through Chapters 2 to 7. Here the thread is error, and where you decide to put it.

Glossary: Rendering: Putting the Error Where You Want It

Terms introduced in this chapter

Error routing — The framing of this chapter: every rendering style is a policy for where unavoidable error is sent — in space, in time, or into structure. Choosing a look is choosing a routing policy.

Temporal coherence — Whether a per-frame decision stays consistent across frames. Independent per-frame dithering is spatially optimal and temporally terrible: the error is white in t , which reads as boiling.

Spatiotemporal blue noise — Noise shaped to be blue in (x, y, t) jointly rather than per-frame. Wolfe et al. (2022). The correct generalisation, and it costs only a precomputed 3D mask.

Dither swim — The artefact where a screen-space threshold field stays fixed while the image moves through it, so texture appears to slide across surfaces. The characteristic failure of naive one-bit rendering in motion.

Object-space anchoring — Indexing the threshold field by surface coordinates rather than screen position, so the pattern travels with the geometry. *Return of the Obra Dinn's* solution, and the general fix for swim.

Advection — Transporting a field along a velocity field. Used here to carry a dither pattern with optical flow so it neither swims nor boils — and, abused deliberately, it is what datamoshing does.

Datamosh — Removing I-frames so that P-frame motion vectors advect the wrong reference texture. Mechanically it is error diffusion with a deliberately mismatched kernel: the transport survives, the content does not.

Motion vector — Per-block displacement stored in an interframe codec. Free, coarse optical flow — available in the bitstream without running an estimator.

Gamma — The nonlinear encoding of luminance. Dithering in gamma-encoded space rather than linear light is the single most common cause of dithered output reading too dark.

Posterisation — Banding from insufficient quantisation levels. The artefact dithering exists to prevent, and occasionally the artefact you want.

Tonal continuity constraint — In tile-based rendering, the requirement that adjacent tiles agree in average density. This is what makes a dither tileset a WFC adjacency rule set (Ch. 6).

Master Glossary

All terms from every chapter, alphabetised. The bracketed number gives the chapter in which the term is introduced.

O

0-form / 1-form [2] — A 0-form is a function on vertices ($f : V \rightarrow \mathbb{R}$); a 1-form is a function on edges. **B** maps 0-forms to 1-forms.

A

Adjacency matrix **A** [2] — $A_{ij} = 1$ (or w_{ij}) when $i \sim j$, else 0.

Advection [9] — Transporting a field along a velocity field. Used here to carry a dither pattern with optical flow so it neither swims nor boils — and, abused deliberately, it is what datamoshing does.

Algebraic connectivity (Fiedler value) [3] — λ_2 , the smallest non-zero Laplacian eigenvalue. Zero iff the graph is disconnected; its eigenvector (the Fiedler vector) gives the standard spectral bisection.

Answer set programming (ASP) [5] — Declarative logic programming over stable models, with `clingo` the standard solver. Strong native support for reachability and cardinality constraints, which makes it the best-fit solver for square Kufic.

Arc consistency [5] — The condition that every value in a variable's domain has a supporting value in each neighbour's domain. Enforced by AC-3. Purely *local* — which is why it cannot see connectivity.

Arity [1] — The number of columns n in a relation. A graph is exactly the arity-2 case; everything distinctive about graph databases follows from fixing $n = 2$.

Attention as adjacency [4] — $\text{softmax}(QK^T / \sqrt{d})$ is a row-stochastic weighted adjacency matrix over tokens, recomputed per layer and per head. A transformer layer is a GNN on a complete digraph.

Autoregressive objective [4] — Maximise $\sum_t \log P(x_t \mid x_{<t})$. Pretraining is this and nothing else; there are no input/output pairs at this stage.

B

Band-limited signal [3] — A signal supported on the low- λ eigenvectors, i.e. smooth with respect to the graph. The assumption underlying graph sampling theory and most GNN inductive bias.

Banna'i / square Kufic [5] — The architectural brick-lattice style in which stroke width and gap width are equal to one module and all strokes are axis-aligned on a grid.

Bayer matrix [8] — The recursively constructed ordered-dither threshold matrix, $M_{2^n} = \begin{pmatrix} 4M_n & 4M_n+2 \\ 4M_n+3 & 4M_n+1 \end{pmatrix}$. Self-similar, which is exactly why its spectrum is spiky.

Belief propagation (BP) [6] — Message passing that computes (approximate) marginals on a factor graph. Running loopy BP instead of WFC gives better-distributed samples at materially higher cost.

Blue noise [8] — Noise whose power spectrum rises with frequency, i.e. energy concentrated above the band the eye resolves. The target distribution for any perceptually good dither.

Broad-nib model [5] — The generative model of blackletter: a rectangular pen held at fixed angle ($\approx 30\text{--}40^\circ$) and swept along a path. Stroke weight is a *consequence* of direction, not a parameter.

C

Cardinality [1] — The number of tuples in a relation, or the number of distinct values in a column. The primary input to a relational cost model, and the reason relational planners degrade on deep traversals: cardinality estimates compound multiplicatively with join depth.

Causal mask [4] — Setting attention logits to $-\infty$ above the diagonal, which prunes the complete digraph to a DAG respecting sequence order.

Cellular automaton (CA) [8] — A graph with a state per vertex updated synchronously by a local rule over its neighbourhood. Formally a

discrete-time, discrete-state message-passing network — the same object as Ch. 2’s GNN, with a lookup table instead of learned weights.

Chebyshev approximation [3] — Approximating $g(\lambda)$ by a degree- K polynomial in $\mathbf{L}$. Because $\mathbf{L}^K$ has support exactly on the K -hop neighbourhood, polynomial order equals spatial locality — and the eigendecomposition is avoided entirely (ChebNet, Defferrard 2016).

Circulant matrix [3] — A matrix in which each row is a cyclic shift of the previous one. Every circulant is diagonalised by the DFT basis — the algebraic reason the classical Fourier transform is the graph Fourier transform on a ring.

Collapse [6] — Assigning one value to a cell, usually sampled from the remaining domain weighted by tile frequency.

Connectivity constraint [5] — The requirement that a letter’s ink cells form one connected component. A *global* constraint: no amount of local propagation implies it. Encoded in SAT via single-source flow or reachability levels.

Constraint satisfaction problem (CSP) [5] — Variables with finite domains, plus constraints restricting joint assignments. The Kufic layout problem is a CSP on a grid graph with domain {ink, void}.

Context window [4] — The maximum token count attended over. Bounds the diameter of the token graph, and drives the quadratic $O(n^2)$ cost of a complete adjacency.

Contradiction [6] — A cell whose domain becomes empty. Because WFC enforces only arc consistency and not path consistency, contradictions are reachable; the reference implementation restarts from scratch rather than backtracking.

Contrast sensitivity function (CSF) [8] — The eye’s spatial-frequency response: band-pass, peaking near 3–5 cycles per degree and falling off sharply above roughly 30. The transfer function that dithering is engineered against.

Cooking [7] — TouchDesigner’s term for evaluating a node. Cooking is *pull-based*: a node cooks only when something downstream requests its output and it is marked dirty.

Cooley–Tukey [3] — The $O(n \log n)$ FFT, which recursively exploits the subgroup lattice of $\mathbb{Z}/n\mathbb{Z}$. Its prerequisite is group structure, which a general graph does not have; hence no fast GFT and an $O(n^3)$ eigendecomposition.

Cutoff frequency [7] — The -3 dB point of a low-pass filter. Fixed low cutoff means smooth but laggy; fixed high cutoff means responsive

but jittery. The One Euro filter refuses the choice by making it a function of speed.

D

Dataflow graph [7] — A program represented as nodes (operations) and edges (data dependencies). TouchDesigner, Houdini, vvvv, Max and Pure Data are all instances.

Datamosh [9] — Removing I-frames so that P-frame motion vectors advect the wrong reference texture. Mechanically it is error diffusion with a deliberately mismatched kernel: the transport survives, the content does not.

Degree matrix $\mathbf{D}$ [2] — Diagonal matrix with $D_{ii} = \deg(i)$.

Diffusion map [2] — Coifman–Lafon embedding using scaled Laplacian eigenvectors as coordinates, so that Euclidean distance in the embedding approximates diffusion distance on the manifold.

Dirty propagation [7] — Marking a node and its dependents stale when a parameter or input changes, so the next frame recomputes only the affected subgraph.

Discrete exterior calculus (DEC) [2] — The framework giving a graph or simplicial complex a native differential calculus: k -forms as functions on k -simplices, with a coboundary operator playing the role of d . Graph calculus is the 0- and 1-form fragment.

Dispersion relation [8] — Growth rate $\sigma(k)$ as a function of wavenumber. On a graph the substitution is $k^2 \rightarrow \lambda_{\text{Laplacian}}$, so the selected pattern is literally the Laplacian eigenvector with the largest growth rate.

Dither [8] — Noise added *before* quantisation. Counter-intuitively it improves the result, because it decorrelates the error from the signal, converting structured banding into noise that can then be spectrally shaped.

Dither swim [9] — The artefact where a screen-space threshold field stays fixed while the image moves through it, so texture appears to slide across surfaces. The characteristic failure of naive one-bit rendering in motion.

DPO [4] — Direct Preference Optimisation: a closed-form reparameterisation that optimises the RLHF objective directly on preference triples, with no explicit reward model or rollouts.

Dwell time [7] — A required minimum duration in a state before a transition commits. Combined with hysteresis, this is what makes mid-air pinch detection usable.

E

Eigenbasis [3] — A basis of eigenvectors of an operator. Diagonalising the operator in this basis is the entire content of ‘working in the frequency domain’.

Entropy heuristic [6] — Choosing the cell with the fewest remaining options (weighted by tile probabilities). Identical to the classical *minimum remaining values* variable-ordering heuristic.

Error diffusion [8] — Quantise a pixel, then push the resulting error into not-yet-visited neighbours according to a kernel. A recursive filter on a directed graph; the recursion is what high-pass shapes the error.

Error routing [9] — The framing of this chapter: every rendering style is a policy for where unavoidable error is sent — in space, in time, or into structure. Choosing a look is choosing a routing policy.

Explicit Euler [2] — First-order time integration $u^{(k+1)} = u^{(k)} + \Delta t \cdot \dot{u}$. One round of graph message passing is exactly one explicit Euler step of graph diffusion.

F

Factor graph [6] — A bipartite graph of variables and factors encoding a joint distribution. WFC is recoverable as a greedy, hard-decision approximation to belief propagation on one.

Feedback TOP [7] — The explicit one-frame delay node that makes recurrence legal in an otherwise acyclic graph. Any cyclic dataflow must be broken by an explicit delay.

Finite state automaton (FSA) [5] — A labelled directed graph with designated start and accepting states. Arabic contextual joining is a deterministic FSA over four positional forms.

Floyd–Steinberg [8] — The canonical error-diffusion kernel: 7/16 ahead, then 3/16, 5/16, 1/16 on the row below. Weights sum to 1, so error is conserved rather than created.

G

Gamma [9] — The nonlinear encoding of luminance. Dithering in gamma-encoded space rather than linear light is the single most common cause of dithered output reading too dark.

Gorilla arm [7] — Fatigue from sustained unsupported mid-air interaction, measurable within minutes. A biomechanical constraint, not a matter of user acclimatisation: design for hands low and at rest, in short bursts.

Graph as program vs graph as data [7] — TouchDesigner’s graph is its *program*; it has no native graph datatype. Graph *structures* live in

a DAT edge list driven from embedded Python. Houdini’s PDG/TOPs, by contrast, exposes an execution dependency graph as manipulable data.

Graph colouring [6] — Assigning labels so adjacent vertices differ. Sudoku is proper 9-colouring of a fixed 81-vertex, 20-regular graph; WFC is one propagation strategy over it.

Graph Fourier transform (GFT) [3] — Eigendecompose $\mathbf{L} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$; then $\hat{f} = \mathbf{U}^\top f$. Columns of $\mathbf{U}$ are the frequency basis, λ_k are squared frequencies.

Graph grammar / shape grammar [5] — A rewriting system of productions $L \rightarrow R$ that match a subgraph and replace it (Stiny, for shapes). The right formalism for encoding script rules: the grammar is the ruleset, the graph is the workpiece.

Graph Laplacian $\mathbf{L}$ [2] — $\mathbf{L} = \mathbf{B}^\top \mathbf{B} = \mathbf{D} - \mathbf{A}$. Composition of discrete divergence with discrete gradient, i.e. $-\nabla^2$. Positive semi-definite by construction, with $\mathbf{L}\mathbf{1} = 0$ always.

H

Halftone [8] — Reproducing continuous tone with a pattern of marks. *AM screening* varies dot size on a fixed grid; *FM screening* varies dot placement at fixed size, which is dithering by another name.

Hand landmark [7] — One of MediaPipe’s 21 keypoints per hand (wrist, plus 4 per digit), returned with normalised image coordinates and a relative depth estimate.

Heat kernel [2] — $e^{-t\mathbf{L}}$, the solution operator of $\dot{u} = -\mathbf{L}u$. Its matrix entries give the amount of heat travelling between two vertices in time t .

Hybrid input [7] — Splitting interaction so the continuous/spatial channel goes to hands and the symbolic/precise channel goes to keyboard or voice. The split that actually ships.

Hypergraph [1] — A generalisation in which an edge may connect any number of vertices. Sits above the graph in the containment lattice and recovers n -ary relations without reification.

Hysteresis [7] — Using different thresholds for entering and leaving a state, so a signal hovering at the boundary cannot chatter. Mandatory for any discrete gesture detected from a continuous quantity.

I

Incidence matrix $\mathbf{B}$ [2] — The $|E| \times |V|$ matrix with -1 at the tail and $+1$ at the head of each edge. It is the discrete exterior derivative: $\mathbf{B}f$ maps a function on vertices to its differences along edges. The discrete gradient.

Index-free adjacency [1] — Storing each edge as a direct pointer from one record to its neighbour, so traversal costs $O(\text{degree})$ rather than $O(\log |V|)$ per hop. The single property that distinguishes a native graph engine from a graph API over tables.

J

Jitter [7] — High-frequency noise in tracked landmark position, present even when the hand is still. A signal-processing problem, not a tracking-quality problem.

Join [1] — The relational operation combining tuples across relations on a shared attribute. A k -hop traversal expressed relationally is a k -way self-join.

Joining form [5] — One of *isolated*, *initial*, *medial*, *final*. Which form a letter takes is determined by its neighbours and by whether each is a joining letter — purely local, hence expressible as an FSA transition.

JSONL [4] — Newline-delimited JSON. A *storage container* for shards, not a training structure — the payload is typically one plain text field. Confusing the container with the objective is the origin of the ‘paired JSON’ misconception.

L

Laplace–Beltrami operator [2] — The generalisation of ∇^2 to a Riemannian manifold. Belkin–Niyogi: the graph Laplacian of a Gaussian-weighted k -NN graph on sampled points converges to it as $n \rightarrow \infty$ with bandwidth $\rightarrow 0$.

Loss masking [4] — Zeroing the loss on prompt/user tokens so gradients flow only from assistant tokens. This — not the JSON — is what makes SFT behave like supervised learning while remaining next-token prediction underneath.

L-system [6] — A parallel string- or graph-rewriting system with an interpretation step. The other major procedural-generation family, and equally a graph grammar.

M

METAFONT model [5] — Knuth’s parametric system in which glyphs are pen paths with constraints solved as a sparse linear system over point variables. Blackletter is the script where this model is most faithful, because the pen mechanics fully determine the form.

Mission graph / space graph [6] — Dormans’ two-stage dungeon generation: generate an abstract graph of gameplay dependencies, then a separate grammar realises it as geometry. Separating what must happen from where it happens.

Mode error [7] — Acting in the belief that the system is in a different state. Endemic to mid-air input because there is no haptic feedback to signal which mode the hand is in.

Model synthesis [6] — Merrell’s 2007 algorithm, predating WFC and in the same family, generating from an example model under adjacency constraints with an explicit modification-in-blocks scheme.

Module [5] — The unit cell of the Kufic grid. Style is defined by the constraint that ink width and void width are both exactly one module.

Moiré [8] — Beat frequencies from superimposed periodic patterns — aliasing, in the sense of Ch. 3. The reason process colour uses screen angles of 15° , 45° , 75° and 0° .

Motion vector [9] — Per-block displacement stored in an interframe codec. Free, coarse optical flow — available in the bitstream without running an estimator.

N

Non-joining letter [5] — The set alif, dal, dhal, ra, zay, waw (alif, dal, dhal, ra, zay, waw and relatives) which never connect to the following letter. They force a break in the connected run — the primary source of the FSA’s branching.

Normalised Laplacian [2] — $\mathbf{L}_{\text{sym}} = \mathbf{D}^{-1/2} \mathbf{L} \mathbf{D}^{-1/2}$, with spectrum confined to $[0, 2]$. Preferred when degrees are heterogeneous, since the unnormalised spectrum otherwise scales with the maximum degree.

O

Object-space anchoring [9] — Indexing the threshold field by surface coordinates rather than screen position, so the pattern travels with the geometry. *Return of the Obra Dinn*’s solution, and the general fix for swim.

One Euro filter [7] — Casiez et al. (CHI 2012): a first-order low-pass filter whose cutoff frequency rises with estimated speed, giving smoothing at rest and responsiveness in motion. Two parameters (f_{cmin} , β); roughly thirty lines of code.

One-bit graphics [8] — Rendering to a two-level display. A constraint that forces every tonal decision into the spatial domain — which is why it produces such a strong and recognisable aesthetic.

Operator families [7] — TOP (2D texture, GPU), CHOP (channel/time-series), SOP (3D geometry), DAT (text and tables), MAT (material), COMP (component). Wire types are family-typed; conversion needs an explicit bridge operator.

Ordered dither [8] — Thresholding against a fixed tiled matrix. Stateless, parallel, and temporally stable — but its error sits at the tile’s harmonics, so the texture is visible as structure rather than grain.

Oversmoothing [2] — The failure mode of deep message-passing networks: repeated application of a diffusion operator converges towards $\ker \mathbf{L}$, which on a connected graph is the constant function. Representations become indistinguishable. Not a bug in the architecture — it is the heat equation reaching equilibrium.

P

Part graph [5] — A graph whose nodes are reusable glyph components (stem, diamond terminal, foot, hairline) and whose edges are attachment constraints. Textura’s illegibility is direct evidence that its part graph is small and heavily reused.

Path consistency [6] — The stronger condition on triples of variables. Enforcing it would eliminate most contradictions at substantially higher cost — the trade WFC declines to make.

Positional encoding [4] — Injected position information. Sinusoidal PEs are the Fourier basis of a path graph; the topology-general version is Laplacian eigenvector PE, used by graph transformers.

Posterisation [9] — Banding from insufficient quantisation levels. The artefact dithering exists to prevent, and occasionally the artefact you want.

Preference triple [4] — (prompt, y_{chosen} , y_{rejected}) The unit of DPO and reward model training. Triples, not pairs.

Propagation [6] — Removing now-unsupported values from neighbouring domains and recursing. Gumin’s implementation maintains per-direction support counts; this is AC-3.

Property graph [1] — The Neo4j/TinkerPop model: nodes and edges both carry a label and an arbitrary key–value map. Distinct from RDF, where attributes are themselves edges.

Q

Quantisation [8] — Mapping a continuous value to one of finitely many levels. Information is destroyed; the only remaining decision is *where the resulting error goes* in frequency space.

R

RDF triple [1] — A statement of the form (subject, predicate, object). A graph edge with a typed label; the storage format that makes reification the native representation.

Reaction–diffusion [8] — $\partial_t u = D \nabla^2 u + f(u, v)$: diffusion plus local nonlinear reaction. On a graph, $\nabla^2 \rightarrow -\mathbf{L}$, so it is Ch. 2’s heat equation with a source term.

Recursive CTE [1] — SQL’s WITH RECURSIVE construct, added in SQL:1999 specifically to recover transitive closure. Semantically sufficient, operationally poor: the planner generally cannot push predicates through the recursion.

Regular path query (RPQ) [1] — A query matching paths whose edge-label sequence belongs to a regular language, e.g. `:knows*` in Cypher or a SPARQL property path. RPQs express transitive closure natively; this is a genuine expressiveness gap, not a performance one.

Reification [1] — Rewriting an n -ary tuple as a node plus n binary edges to its attribute values. The mechanism by which any relational schema can be re-expressed as a graph, at the cost of moving arity out of the schema and into the topology.

Relation [1] — A subset of a Cartesian product $R \subseteq A_1 \times \dots \times A_n$. The whole of the relational model is relations plus operations on them.

S

Self-avoiding walk [5] — A path in a graph that revisits no vertex. Embedding a letter sequence into the Kufic grid subject to spacing constraints is a constrained self-avoiding walk problem, NP-hard in general.

Serpentine scan [8] — Alternating scan direction each row. Costs one line of code and removes the directional worm artefacts that a uniform left-to-right raster produces.

Skeleton and envelope [5] — The skeleton is the path the pen travels; the envelope is the outline swept out. Blackletter is naturally authored as skeleton + nib, and only rasterised at the end.

Spatiotemporal blue noise [9] — Noise shaped to be blue in (x, y, t) jointly rather than per-frame. Wolfe et al. (2022). The correct generalisation, and it costs only a precomputed 3D mask.

Spectral filter [3] — A function $g(\lambda)$ applied to the eigenvalues: $g(\mathbf{L})f = \mathbf{U}g(\Lambda)\mathbf{U}^T f$. Graph convolution is *defined* this way, so the convolution theorem holds by construction.

Spectral gap [3] — $\lambda_2 - \lambda_1$. Controls mixing time of random walks and the convergence rate of diffusion; large gap means fast homogenisation, hence rapid oversmoothing.

Stencil [2] — The local pattern of neighbours used by a finite-difference scheme (the 5-point Laplacian, say). The stencil defines the edge set of the induced grid graph.

Stiffness matrix [2] — The sparse matrix assembled by a finite element discretisation. Its sparsity pattern is literally an adjacency matrix over the mesh; FEM assembly is graph construction.

Superposition [6] — WFC's term for a cell's remaining domain, i.e. the set of still-admissible tiles.

Supervised fine-tuning (SFT) [4] — Training on role-annotated conversations. Superficially paired, but flattened to a single token stream with role-delimiter tokens before training.

T

Temporal coherence [9] — Whether a per-frame decision stays consistent across frames. Independent per-frame dithering is spatially optimal and temporally terrible: the error is white in t , which reads as boiling.

Tokenisation [4] — Mapping text to integers via a learned subword vocabulary (BPE, unigram). The model never sees characters, let alone file formats.

Tonal continuity constraint [9] — In tile-based rendering, the requirement that adjacent tiles agree in average density. This is what makes a dither tileset a WFC adjacency rule set (Ch. 6).

Totalistic rule [8] — A CA rule depending only on the *sum* of neighbour states, not their arrangement. Permutation-invariant aggregation — precisely the constraint that defines a graph neural

network layer.

Transitive closure [1] — Given relation E , the relation E^+ containing (u, v) whenever a path $u \rightarrow v$ exists. Provably *not* expressible in first-order logic, hence not in core relational algebra.

Turing pattern [8] — Stationary spatial structure arising when diffusion *destabilises* a uniform state. Requires the inhibitor to diffuse faster than the activator.

U

Uncertainty principle [3] — A signal cannot be arbitrarily concentrated in both vertex and spectral domains. The reason a sharp spectral filter is spatially delocalised and a K -hop filter cannot be spectrally sharp.

V

Void-and-cluster [8] — Ulichney's method for generating blue-noise masks: iteratively find the largest void and tightest cluster and swap. An explicit optimisation over a spatial objective — constraint satisfaction, not a formula.

W

Wave Function Collapse (WFC) [6] — Gumin's generation algorithm: maintain a domain per cell, repeatedly collapse the minimum-entropy cell and propagate. Despite the name it contains no physics — it is arc consistency with a heuristic.

Sources and Further Reading

Grouped by chapter, and annotated — the annotation is the useful part. Where a single work would change how you think about a whole area, it is marked ►.

1 Databases

- **Codd**, *A Relational Model of Data for Large Shared Data Banks* (1970). Worth reading in the original for how explicitly it is a paper about *physical data independence* — the argument is that the access path should not be visible in the query, which is precisely the thing graph databases partially give back.
- **Angles & Gutierrez**, *Survey of Graph Database Models* (2008). The clearest account of where the models genuinely differ rather than merely differing in marketing.
- ► **Aho & Ullman**, *Universality of Data Retrieval Languages* (1979). The source of the transitive-closure result. If you want the expressiveness gap as a theorem rather than an assertion, it is here.

2 The Laplacian and PDEs

- ► **Chung**, *Spectral Graph Theory* (1997). The standard reference. Short, dense, and the normalised-Laplacian conventions in it are the ones everyone else inherits.
- **Grady & Polimeni**, *Discrete Calculus* (2010). The book that takes the $\mathbf{B}^T \mathbf{B}$ view seriously and builds it out properly. The best single source for the framing used in Chapter 2.
- **Belkin & Niyogi**, *Laplacian Eigenmaps and Spectral Techniques for Embedding and Clustering* (2003), and *Towards a Theoretical Foundation for Laplacian-Based Manifold Methods* (2005). The second is where the convergence result actually lives.
- **Coifman & Lafon**, *Diffusion Maps* (2006).
- **Chamberlain et al.**, *GRAND: Graph Neural Diffusion* (2021). Makes the GNN-as-PDE-solver identification explicit and then does something useful with it.
- **Trottenberg, Oosterlee & Schüller**, *Multigrid* (2000). Read the coarsening chapters as graph coarsening and the connection to hierarchical graph pooling is immediate.

3 Fourier and spectral methods

- ► **Shuman et al.**, *The Emerging Field of Signal Processing on Graphs* (2013). The single best entry point to graph signal processing, and the source of the classical/graph dictionary in Chapter 3.
- **Defferrard, Bresson & Vandergheynst**, *Convolutional Neural Networks on Graphs with Fast Localized Spectral Filtering* (2016). ChebNet — where polynomial order equals hop

radius is used deliberately.

- **Van Loan**, *Computational Frameworks for the Fast Fourier Transform* (1992). If you want the group-theoretic reason the FFT is fast rather than the recursion as a trick.
- **Bronstein, Bruna, Cohen & Veličković**, *Geometric Deep Learning* (2021). The “blueprint” framing: symmetry determines architecture. Directly generalises the argument of Chapter 3.

4 Language models

- **Vaswani et al.**, *Attention Is All You Need* (2017).
- ► **Joshi**, *Transformers are Graph Neural Networks* (2020). Short, and makes the identification in Chapter 4 cleanly and without overclaiming.
- **Dwivedi & Bresson**, *A Generalization of Transformer Networks to Graphs* (2020). Where Laplacian eigenvector positional encodings are introduced.
- **Lim et al.**, *Sign and Basis Invariant Networks* (2022). SignNet/BasisNet — the principled fix for the eigenvector ambiguity flagged in Chapter 4.
- **Rafailov et al.**, *Direct Preference Optimization* (2023). The triples, and why they are triples.
- **Zaheer et al.**, *Big Bird* (2020). Sparse attention read explicitly as graph construction, including the spectral-gap argument.

5 Script and type

- ► **Knuth**, *The METAFONTbook* (1986). Still the best exposition of the pen-and-skeleton model. Read the pen chapters even if you never write a line of METAFONT.
- **Stiny**, *Introduction to Shape and Shape Grammars* (1980). The formalism behind Figure 5.5.
- **Necipoğlu**, *The Topkapı Scroll: Geometry and Ornament in Islamic Architecture* (1995). The primary-source account of how these patterns were actually constructed, including the modular grid logic.
- **Gebser, Kaminski, Kaufmann & Schaub**, *Answer Set Solving in Practice* (2012). The `clingo` book. The reachability and cardinality constructions are exactly what the Kufic connectivity problem needs.
- **Ehrig et al.**, *Fundamentals of Algebraic Graph Transformation* (2006). The rigorous treatment of $L \rightarrow R$ productions.

6 Procedural generation

- ► **Karth & Smith**, *WaveFunctionCollapse is Constraint Solving in the Wild* (2017). The paper that names the connection properly. Read it before writing another propagator.
- **Gumin**, *WaveFunctionCollapse* (2016), source repository. The reference implementation is small enough to read end to end, and the support-counting propagator is the part to study.

- **Merrell**, *Example-Based Model Synthesis* (2007). Predates WFC, same family, and the modification-in-blocks scheme is the better answer for large outputs.
- **Mackworth**, *Consistency in Networks of Relations* (1977). AC-3 in the original. Startlingly readable.
- **Russell & Norvig**, *Artificial Intelligence: A Modern Approach*, constraint satisfaction chapter. MRV, degree heuristic, backjumping — the standard machinery WFC mostly does not use.
- **Dormans**, *Adventures in Level Design: Generating Missions and Spaces for Action Adventure Games* (2010).
- **Prusinkiewicz & Lindenmayer**, *The Algorithmic Beauty of Plants* (1990). Free online, and the canonical L-system text.

7 Dataflow and interaction

- ► **Casiez, Roussel & Vogel**, *1 ∈ Filter: A Simple Speed-based Low-pass Filter for Noisy Input in Interactive Systems* (CHI 2012). Thirty lines of code and a genuinely good paper. The reference implementation on the authors' page is complete.
- **Zhang et al.**, *MediaPipe Hands: On-device Real-time Hand Tracking* (2020).
- **Hincapié-Ramos et al.**, *Consumed Endurance: A Metric to Quantify Arm Fatigue of Mid-Air Interactions* (CHI 2014). Gorilla arm made measurable, which is what lets you design against it rather than hope.
- **Buxton**, *Chunking and Phrasing and the Design of Human-Computer Dialogues* (1986). The theoretical basis for the hybrid-input split in Figure 7.5, forty years early.
- **Johnston et al.**, *Advances in Dataflow Programming Languages* (2004). The survey that situates TouchDesigner, Houdini, Max and vvvv in one framework.

8 Dithering, halftones and automata

- ► **Ulichney**, *Digital Halftoning* (1987), and *The Void-and-Cluster Method for Dither Array Generation* (1993). The standard reference and the standard algorithm. The 1993 paper is short and is the one to implement.
- **Floyd & Steinberg**, *An Adaptive Algorithm for Spatial Greyscale* (1976). Two pages. Worth reading for how little is claimed.
- **Bayer**, *An Optimum Method for Two-Level Rendition of Continuous-Tone Pictures* (1973). The recursive matrix, and the optimality argument — which is genuinely optimal, but for the wrong objective.
- **Lau & Arce**, *Modern Digital Halftoning* (2008). The signal-processing treatment; the delta-sigma connection made explicit.
- ► **Turing**, *The Chemical Basis of Morphogenesis* (1952). Still the clearest statement of the mechanism, and startlingly modern in its treatment of the linear stability analysis.
- **Murray**, *Mathematical Biology II: Spatial Models* (2003). The full dispersion-relation apparatus behind Figure 8.6.

- **Wolfram**, *A New Kind of Science* (2002). Read selectively; the elementary-CA catalogue is genuinely useful reference material regardless of what one makes of the surrounding claims.
- **Mordvintsev, Randazzo, Niklasson & Levin**, *Growing Neural Cellular Automata* (Distill, 2020). Where the CA/GNN identification stops being an observation and becomes a method.
- **Chan**, *Lenia: Biology of Artificial Life* (2019). The continuous-state generalisation.

9 Rendering video and data

- ► **Wolfe, Morrical, Akenine-Möller & Ramamoorthi**, *Spatiotemporal Blue Noise Masks* (EGSR 2022). The correct answer to the temporal problem, with code. Same runtime cost as an ordered dither.
- **Georgiev & Fajardo**, *Blue-noise Dithered Sampling* (2016). The rendering-side treatment; where the golden-ratio offset trick is formalised.
- **Pope**, *Return of the Obra Dinn* — development posts on the TIGSource forums (2014–2018). The most useful practical account of one-bit rendering in motion that exists, including several dead ends. Object-space anchoring is worked out in public over several posts.
- **Menkman**, *The Glitch Moment(um)* (2011). The theoretical treatment of deliberate error, and the source of the useful artefact/glitch distinction.
- **Richardson**, *The H.264 Advanced Video Compression Standard* (2010). If you want to datamosh deliberately rather than by trial and error, the motion-vector and reference-frame chapters are what you need.
- **Fairchild**, *Color Appearance Models* (2013). For the gamma and tone-mapping stages of Figure 9.3, and for why “perceptually uniform” is a stronger claim than it sounds.

Colophon. Set in Palatino with Helvetica for headings. Typeset with L^AT_EX; all 37 figures are vector graphics drawn in TIKZ and PGFPLOTS, with no raster images. The numerical figures — diffusion on the ring, the ring spectrum, the path-graph modes, the One Euro comparison, the four dithered ramps, the error spectra and the Turing dispersion relation — are computed by the accompanying `gen_figs.py` and `gen_dither.py` rather than sketched, so the values printed in the captions are the values plotted. The glossary is generated from a single structured source, which is why the same term is defined identically in its chapter and in the master list.